

An Attack to Universally Composable Commitments from Malicious Physically Uncloneable Functions and how to Avoid it

Lourenço Abecasis^{1,2}, Paulo Mateus^{1,2}, and Chrysoula Vlachou^{1,2}

¹ Instituto de Telecomunicações

Av. Rovisco Pais 1, 1049-001 Lisboa, Portugal

² Department of Mathematics, Instituto Superior Técnico

Universidade de Lisboa, Av. Rovisco Pais 1, 1049-001 Lisboa, Portugal

Abstract. In this work, we explore the possibility of unconditionally secure universally composable (UC) commitments, a very relevant cryptographic primitive in the context of secure multi-party computation. To this end, we assume the existence of Physically Uncloneable Functions (PUFs), a hardware security assumption that has been proven useful for securely achieving diverse tasks. In prior work [ASIACRYPT 2013, LNCS, vol. 8270, pp. 100–119] it was shown that a protocol for unconditional UC-secure commitments can be constructed even when the PUFs are malicious. Here, we report an attack to this protocol, as well as a few more issues that we identified in its construction. To address them, first we revise some of the previous PUF properties, and introduce new properties and tools that allow us to rigorously develop and present the security proofs. Second, we propose two different ways for making the commitment scheme secure against the attack we found. The first involves considering a new model where the creator of a PUF is notified whenever the PUF is queried and the second involves restricting adversaries to only being able to create stateless malicious PUFs. Finally, we analyze the efficiency of our schemes and show that our constructions are advantageous in this respect compared to the original proposal.

Keywords: Secure multi-party computation · Universally composable security · Physically uncloneable functions

1 Introduction

The universally composable (UC) security framework, introduced in [1], is a powerful standard ensuring that cryptographic protocols remain secure when they are composed with other protocols and/or instances of themselves. UC-security has been extensively explored in the context of secure multi-party computation (MPC) [2–8], and it has been shown that in the plain model, where no additional assumptions are made, UC-secure MPC is impossible [3]. Therefore, to achieve this compelling composability property one needs to make further assumptions, see e.g. access to a common reference string [3]. A recent line of successive works [10–15] study the possibility of constructing UC-secure protocols for MPC primitives assuming Physically Uncloneable Functions (PUFs). The existence of PUFs is a hardware assumption, i.e. it is based on the physical properties of specific hardware components. PUFs were introduced in [9] and are devices produced by a complex physical manufacturing process, making it practically infeasible to clone. A PUF can be evaluated by providing a physical stimulus, called the *challenge*, to which it responds with a noisy output, called the *response*. Due to their uncloneability, PUFs were initially used as hardware tokens for device identification and authentication, however additional security properties have been considered and to date there is a vast bibliography on how to incorporate them in diverse security applications (see e.g. [16] for a review).

1.1 Prior work on UC-secure MPC using PUFs

Focusing on the works concerning the use of PUFs for UC-secure MPC, the first is the one by Brzuska et al. [10], where UC-secure protocols for oblivious transfer (OT), bit commitment and key exchange were proposed. The PUFs considered therein are assumed to be *trusted*, i.e. they have been produced through the prescribed manufacturing process and they have not been tampered with by an adversary. This assumption was later lifted in [11], where *malicious* PUFs were introduced to account for adversaries that can create PUFs with arbitrary malicious behaviour. A malicious PUF, in general, is a hardware token that meets the syntactical requirements of an honest PUF. It could be a fake PUF, possibly programmed with malicious code, or a PUF whose output on some input might depend on previous inputs. The latter is called a *stateful* PUF and is in contrast with *stateless* PUFs. While honest PUFs are necessarily stateless, malicious PUFs can be either stateful or stateless. In this model of stateful malicious

PUFs, and assuming that a malicious PUF cannot interact with its creator once it is sent away to another party, Ostrovsky et al., developed a computationally UC-secure commitment scheme [11] and a commitment scheme with unconditional indistinguishability-based security. They also proposed an unconditionally UC-secure OT protocol in the so-called *oblivious-query model*, where the adversaries cannot create malicious PUFs, but they can query trusted PUFs via non-prescribed processes. Furthermore, they showed that UC-security is impossible when considering adversaries from both the malicious PUFs and the oblivious-query models. Subsequently, Damgård and Scafuro constructed an unconditionally UC-secure commitment scheme [12] using the malicious PUFs model from [11], and they showed that these commitments are unconditionally UC-secure in the stateless tamper-proof hardware token model [17]. Following this, Dachman-Soled et al. derived two important results [13]: the first is the impossibility of unconditionally secure OT, for both stand-alone and indistinguishability-based security, in the stateful malicious PUF model, and the second is the possibility of UC-secure OT in the stateless malicious PUF model. In all the works mentioned so far, malicious PUFs are assumed to maintain a priori unbounded states, however Badrinarayanan et al. argued that this is a very strong assumption that might be practically irrelevant [14]. Therefore, they modified the adversarial model such that the malicious stateful PUF can maintain an a priori bounded state, and showed that unconditional UC-secure computation of any functionality is possible in this model, employing the construction from [12]. Moreover, they introduced a new model, where the adversary can generate malicious stateless PUFs and encapsulate honest PUFs inside them even without the knowledge of the functionality of the inner PUFs. The outer malicious PUF can make oracle calls to the inner PUFs, and an honest party is not able to tell whether they are interacting with an honest PUF or a malicious PUF encapsulating honest ones. In this *malicious encapsulation model*, they showed that unconditional UC-secure computation of any functionality is still possible. Finally, Magri et al. introduced a more general and stronger model, that of *fully malicious hardware tokens*, of which PUFs are a special case [15]. Such tokens do not have a priori bounded states, arbitrary code might be installed inside them, and they can encapsulate and decapsulate other – possibly fully malicious – tokens within themselves. An everlastingly UC-secure commitment scheme was constructed under the Learning With Errors assumption, and it was shown that everlastingly UC-secure OT is impossible using non-erasable honest tokens.

1.2 Our contributions

Motivated by the relevance of unconditionally UC-secure commitments for MPC, we thoroughly studied [12] as the methods used to construct them become very useful in this context. Importantly enough, in the quantum cryptography paradigm, i.e. assuming access to quantum channels, unconditionally UC-secure commitments imply quantum UC-secure OT [18]. In this light, the commitments in [12] enable quantum UC-secure MPC relying solely on physical assumptions. While exploring this work [12], we came across some issues that weaken the construction proposed therein. To obtain a UC-secure protocol for commitments the authors first construct an extractable ideal (i.e., statistically hiding and binding) commitment scheme using the ideal commitment protocol proposed previously in [11]. Then, they develop an unconditional black-box compiler that transforms the extractable ideal commitments to UC-secure commitments.

The first issues we found concern already the starting ideal commitment protocol from [11]. The proofs for both the binding and hiding properties were neither rigorous nor clear and did not account for certain adversarial behaviours. To address these issues, in Section 3, we revise the PUF properties used in [11] and [12], introduce new PUF properties and reformulate the proofs accordingly in a rigorous way. Note that in our work commitments are stated more generally for bitstrings instead of bits, which was the case in previous works.

Second, we noticed that a malicious sender could break the extractability property of the extractable ideal commitment protocol introduced in [12]. In Section 4, we present the attack and propose two different ways of fixing it. The first involves changing the PUF model, by adding to honest PUFs the feature of notifying their creators whenever they are queried; adversaries cannot interfere with these notifications. Our second fix involves changing the protocol, however, this change comes at the cost of having to assume that the malicious PUFs are stateless. Obviously, both fixes we propose weaken the commitment schemes from [12] either by requiring extra properties from the PUFs (PUFs notifying after a query is an assumption that may be unrealistic if an adversary can interfere with or suppress such signals) or by significantly weakening the adversarial model (by assuming stateless PUFs), however these were the only ways we found to make them resist our attack. Importantly, the same approach as in [12] is followed in [14], therefore, the attack that we found and, in turn, the ways to fix it influence accordingly the results in [14].

Finally, we noticed one more issue: the UC-secure commitment protocol involves multiple commitments, and in the UC-security proof it is implicitly assumed that the security properties of these commitments are preserved when they are employed collectively within a more complex protocol. This assumption, though, was not proven. To avoid

possible flaws in the proof and to create a more efficient UC-secure protocol, through the parallelization of these commitments, we generalized the definition of a commitment scheme and its corresponding properties to enable the commitment of many strings at once, and adjusted accordingly the compiler from [12]. This analysis and the updated compiler in our two new models (i.e. when we consider PUFs that notify their creators upon being queried and when PUFs are stateless) is presented in Section 5. Here, we should mention that our notation overall is different from that in previous works, as we wanted it to be generalizable for the purpose of the aforementioned collective commitments scheme. Finally, in Section 6 we present the protocols for unconditional UC-secure commitments in both new models and discuss their efficiency by comparing them also to the efficiency of the original proposal [12].

Even though the protocols we propose face restrictions, hopefully, our contribution will trigger further work on overcoming them, and using our new tools and approach, UC-secure commitment schemes, possibly even more efficient, will be developed in stronger adversarial models with less requirements on the PUFs. At the same time, our observations raise the concern that fixing the problem in the extractable ideal commitment protocol may inherently require moving to significantly less realistic models. If no solution exists within the standard assumptions, then it is possible that this entire line of work has hit a fundamental barrier. In that case, progress may require abandoning this direction altogether in favor of fundamentally different protocol designs.

2 Physically Uncloneable Functions

2.1 PUF families and fuzzy extractors

Following the works mentioned above, we use the definition for a PUF family introduced in [10], and we consider that all adversaries are Probabilistic Polynomial-Time (PPT). A PUF family $\mathcal{P}$ is defined by two stateless probabilistic Turing machines **Sample** and **Eval**, which are not necessarily efficient. The index sampling algorithm, **Sample**, outputs an index id and represents the manufacturing process of a PUF, while the evaluation algorithm, **Eval**, takes a challenge s as input and outputs the corresponding response σ .

Definition 1. Let $\mathcal{P} = (\text{Sample}, \text{Eval})$ be a pair of algorithms that run on security parameter n . We say that $\mathcal{P}$ is a (rg, d_{noise}) -PUF family if it satisfies the following properties:

- **Index sampling:** The sampling algorithm Sample_n outputs an index id from a fixed index set $\mathcal{I}_n$. Each $\text{id} \in \mathcal{I}_n$ corresponds to a set of distributions $\mathcal{D}_{\text{id}}$: for each challenge $s \in \{0, 1\}^n$, $\mathcal{D}_{\text{id}}(s)$ is a distribution on $\{0, 1\}^{rg(n)}$.
- **Evaluation:** For all challenges $s \in \{0, 1\}^n$, the evaluation algorithm $\text{Eval}_n(\text{id}, s)$ outputs a response $\sigma \in \{0, 1\}^{rg(n)}$ according to the distribution $\mathcal{D}_{\text{id}}(s)$.
- **Bounded noise:** For all indices $\text{id} \in \mathcal{I}_n$ and challenges $s \in \{0, 1\}^n$, if σ and σ' are obtained from running $\text{Eval}_n(\text{id}, s)$ twice, then $d(\sigma, \sigma') < d_{\text{noise}}(n)$, where d is the Hamming distance.

The main security property of PUFs, unpredictability, is formalized in [10] using average min-entropy. Let

- $\text{PUF} \leftarrow \mathcal{P}$ mean $\text{id} \leftarrow \text{Sample}_n$ and then restricting the name “PUF” to this id ;
- $\sigma \leftarrow \text{PUF}(s)$ mean $\sigma \leftarrow \text{Eval}_n(\text{id}, s)$;
- $\sigma_{\mathbf{q}} \leftarrow \text{PUF}(\mathbf{q})$ mean $\sigma_{\mathbf{q}} \leftarrow (\text{Eval}_n(\text{id}, q))_{q \in \mathbf{q}}$, where $\mathbf{q}$ is a list of queries.

Definition 2. We say that a (rg, d_{noise}) -PUF family $\mathcal{P} = (\text{Sample}, \text{Eval})$ is $(d_{\min}, m)$ -unpredictable if for any $s \in \{0, 1\}^n$ and list of queries $\mathbf{q}$ of polynomial size in n , the following condition holds:

$$d(\mathbf{q}, s) \geq d_{\min}(n) \implies \tilde{H}_{\infty}(\text{PUF}(s) \mid \text{PUF}(\mathbf{q})) \geq m(n),$$

where $d(\mathbf{q}, s) := \min_i d(q_i, s)$. Such a PUF-family is called a $(rg, d_{\text{noise}}, d_{\min}, m)$ -PUF family.

Since the PUF evaluation is inherently noisy, the fuzzy extractors (FE) from [20] are used in [10] to convert noisy, high-entropy outcomes of PUFs into reproducible random values.

Definition 3. Let $\mathcal{M}$ be a metric space with distance function dis . Consider a pair of efficient randomized algorithms $\text{FE} = (\text{Gen}, \text{Rep})$ such that:

- **Gen**, on input $w \in \mathcal{M}$, outputs a pair (st, p) , where $st \in \{0, 1\}^l$ is called the secret string and $p \in \{0, 1\}^*$ is called the helper data string³;

³ Here, we use Kleene star notation, so $\{0, 1\}^*$ denotes the set of all finite binary strings.

- **Rep**, on input an element $w \in \mathcal{M}$ and a helper data string $p \in \{0, 1\}^*$, outputs a string st .

We say that **FE** is an average-case (m, ℓ, t, ϵ) -FE if:

- **Correctness**: For all $w, w' \in \mathcal{M}$, if $\text{dis}(w, w') \leq t$ and $(st, p) \leftarrow \text{Gen}(w)$, then $\text{Rep}(w', p) = st$;
- **Security**: Consider some random variables W , which takes values $w \in \mathcal{M}$, and **EXTRA**, which corresponds to some additional information related to W . Let D_0 and D_1 denote the distributions of the outputs from the programs depicted in Fig. 1. Then,

$$\tilde{H}_\infty(W \mid \text{EXTRA}) \geq m \implies \text{SD}(D_0, D_1) < \epsilon^4.$$

D_0	D_1
$(w, \text{extra}) \leftarrow (W, \text{EXTRA})$	$(w, \text{extra}) \leftarrow (W, \text{EXTRA})$
$(st, p) \leftarrow \text{Gen}(w)$	$(st, p) \leftarrow \text{Gen}(w)$
return (st, p, extra)	$u \leftarrow \{0, 1\}^\ell$
	return (u, p, extra)

Fig. 1. Programs for the security property of FEs.

We refer to average-case FEs simply as FEs. Now, consider the functions m, ℓ, t and ϵ and algorithms (Gen, Rep) that run on the security parameter n . We say that **FE** = (Gen, Rep) is a (m, ℓ, t, ϵ) -FE family if, for each $n \in \mathbb{N}$, $\text{FE}_n = (\text{Gen}_n, \text{Rep}_n)$ is a $(m(n), \ell(n), t(n), \epsilon(n))$ -FE. The following definition specifies the parameters for a FE family to be matching with a given PUF family.

Definition 4. Let $\mathcal{P}$ be a $(rg, d_{\text{noise}}, d_{\text{min}}, m)$ -PUF family and **FE** be a $(m_{\text{FE}}, \ell_{\text{FE}}, t_{\text{FE}}, \epsilon_{\text{FE}})$ -FE family. We say that $\mathcal{P}$ and **FE** have matching parameters if:

- FE_n is defined on the metric space $\{0, 1\}^{rg(n)}$ with Hamming distance d ;
- $m_{\text{FE}}(n) = m(n)$;
- $t_{\text{FE}}(n) = d_{\text{noise}}(n)$;
- $\epsilon_{\text{FE}}(n) = |\epsilon(n)|^5$.

2.2 Our adversarial model for malicious PUFs

Let us start by briefly describing the malicious PUFs model, as it slightly differs from those in previous works.

Malicious PUFs were first introduced in [11] to model adversaries that can tamper with the manufacturing process of PUFs, potentially embedding additional behaviors, such as query logging, into the PUF tokens. To keep the model as general as possible, [11] places no restrictions on the malicious PUF families other than requiring that they share the same syntax as honest PUF families. In addition, the malicious PUF functionality is parameterized by both an honest and a malicious PUF family. We argue, however, that this approach grants the adversary excessive power due to its lack of specificity. For instance, without restrictions on the malicious PUF family, an adversary could, in an extreme case, create a PUF that replicates the most recently generated honest PUF, which would violate uncloneability – a fundamental property of PUFs. Furthermore, as pointed out in [13], restricting malicious PUFs to a fixed family prevents them from being created adaptively throughout the protocols. To address these concerns, [13] proposes a more explicit model in which malicious PUFs are defined by arbitrary code, potentially including oracle access to a freshly created honest PUF that remains inaccessible to other parties, and we follow this modelling as well. However, in [13] each honest PUF was assumed to generate a random response on the first query for each challenge and return the same response for repeated queries. We believe that this simplification might not be realistic⁶, therefore we chose to retain the original approach for modeling honest PUFs.

⁴ SD denotes the statistical distance.

⁵ Here, $\epsilon(n)$ denotes an arbitrary negligible function. We use this notation throughout our work.

⁶ This kind of consistency is not guaranteed when using fuzzy extractors (see Definition 3): if we query $\text{PUF}(s)$ and receive different responses, σ and σ' , the resulting outputs st and st' from applying **Gen** may not match.

We model malicious PUFs, denoted as MPUF, as follows: each MPUF consists of a finite set⁷ of freshly created honest PUFs, that are inaccessible to other parties and a possibly stateful Turing machine M with oracle access to those PUFs. Querying MPUF on a challenge s thus amounts to querying M on s , which may change M 's state. For security parameter n , the machine M operates with $k_{\text{state}}(n)$ bits of memory. The state size can be:

- **bounded**, in which case it is represented as a function $k_{\text{state}} : \mathbb{N} \rightarrow \mathbb{N}$, or
- **unbounded**, in which case $k_{\text{state}} = \infty$.

The corresponding functionality $\mathcal{F}_{\text{MPUF}}$ is depicted in Fig. 2, and it is parameterized by a PUF family $\mathcal{P}$ (see Definition 1) and a state bound $k_{\text{state}}(n)$. For clarity, we describe the functionality in the case where a malicious PUF has access to a single honest PUF. However, this restriction is made purely for simplicity and the functionality can be naturally extended to support access to multiple honest PUFs.

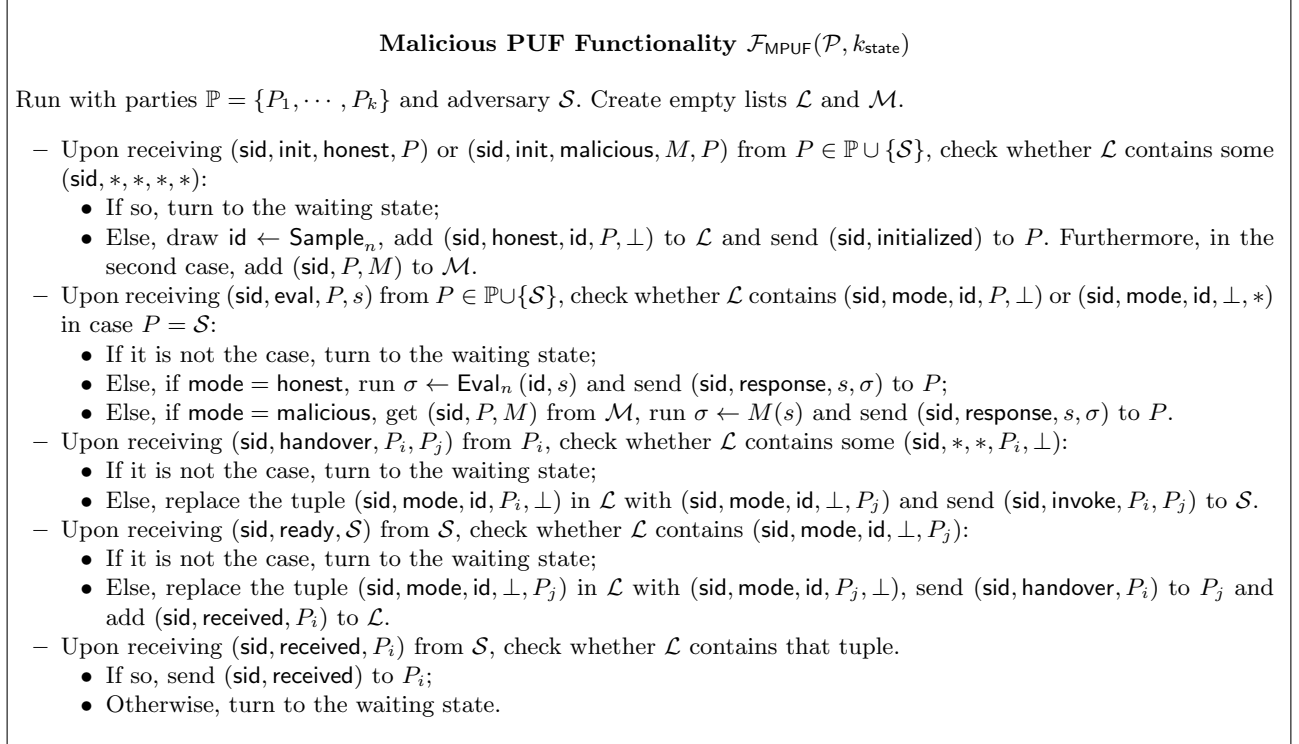


Fig. 2. The malicious PUF functionality.

3 The ideal commitment from [11]

In [12], the ideal commitment⁸ CPUF from [11] (depicted in Fig. 16 in Appendix A), which is essentially based on the one from [19], serves as the starting point for constructing their UC-secure commitment scheme. However, we identified several issues with this construction. In particular, the corresponding hiding proof was not sufficiently rigorous and relied on oversimplified reasoning. This motivated us to refine and extend several PUF properties that are essential for formal security analysis but had previously been overlooked. Additionally, the binding proof lacked clarity, so we adapted it to more closely follow the original argument from [19], generalizing it to account for the inherent noise of PUFs by incorporating suitable entropy assumptions.

3.1 PUF properties: new and revised

In previous work, the security of PUF-based protocols was argued using a small set of properties relating PUFs and fuzzy extractors. Those properties were simple consequences of the definitions we discussed in the previous section

⁷ Since we consider PPT adversaries, the size of this set must be polynomially bounded in the security parameter n .

⁸ See Definition 10 in Appendix A.

and were treated as sufficient for proving security. However, upon closer examination, some subtle adversarial behaviors were overlooked in those arguments. In this section, we revisit those properties with a more rigorous approach. Some are reformulated or strengthened to address gaps in previous analyses, while others are entirely new, introduced to capture aspects of the adversary’s capabilities that were previously ignored. Here, we mostly provide informal overviews and motivation for each property; their formal details are deferred to the corresponding appendices.

The following properties are straightforward consequences of the definitions of PUF and FE families with matching parameters:

Response consistency: Let σ, σ' be responses of PUF when queried twice with the same challenge s . If $(st, p) \leftarrow \text{Gen}(\sigma)$, then $\text{Rep}(\sigma', p) = st$.

Almost-uniformity: Let $s \in \{0, 1\}^n$ and consider the programs depicted in Fig. 3. Then, for each $n \in \mathbb{N}$,

$$\tilde{H}_\infty(\text{PUF}(s) \mid \text{EXTRA}) \geq m(n) \implies \text{SD}(D_0, D_1) < \epsilon_{\text{FE}}(n).$$

D_0	D_1
$\text{PUF} \leftarrow \mathcal{P}$	$\text{PUF} \leftarrow \mathcal{P}$
$\sigma \leftarrow \text{PUF}(s)$	$\sigma \leftarrow \text{PUF}(s)$
$(st, p) \leftarrow \text{Gen}(\sigma)$	$(st, p) \leftarrow \text{Gen}(\sigma)$
$\text{extra} \leftarrow \text{EXTRA}$	$u \leftarrow \mathbb{s} \{0, 1\}^{rg(n)}$
return (st, p, extra)	$\text{extra} \leftarrow \text{EXTRA}$
	return (u, p, extra)

Fig. 3. Programs for the almost-uniformity property.

Notice that if we take the random variable $\text{EXTRA} = \text{PUF}(q)$ with $d(q, s) \geq d_{\min}(n)$, unpredictability ensures $\tilde{H}_\infty(\text{PUF}(s) \mid \text{EXTRA}) \geq m(n)$ and thus almost-uniformity ensures $\text{SD}(D_0, D_1) < \epsilon_{\text{FE}}(n)$. This particular case, known as **extraction independence**, is the approach used in [10]. However, we adopt this more general property, as it offers greater flexibility in security proofs.

Furthermore, an additional property called **well-spread domain** was also proven in [10]. Informally, it states that if an honest party generates a challenge s uniformly at random, then an adversary attempting to choose a challenge “close” to s (that is, within a distance smaller than $d_{\min}$) should only have a negligible probability of success; this was achieved by assuming $d_{\min}(n) \in o(n/\log(n))$. The goal was to use it along with extraction independence to arrive at what we call the **indistinguishability property**: if an honest party generates a challenge s uniformly at random, then the output of the FE applied to the response $\text{PUF}(s)$ should be indistinguishable from a uniformly random response, even if the adversary has access to PUF itself.

However, ensuring that this well-spread domain property still holds when the adversary has access to $\text{PUF}(s)$ – we call this the **close query (CQ) property** – is essential for indistinguishability, and it turns out to be non-trivial, as $\text{PUF}(s)$ may inadvertently reveal some information about s . Since this had not been considered in previous works, instead of $d_{\min}(n) \in o(n/\log(n))$ we had to make the following stronger assumption:

Preimage entropy: Let $\mathcal{P}$ be a PUF family and consider the neighborhood $B_n^d(x) = \{y \in \{0, 1\}^n : d(x, y) < d\}$ around $x \in \{0, 1\}^n$.⁹ Then,

$$\left| B_n^{d_{\min}(n)} \right| 2^{-\tilde{H}_\infty(S \mid \text{PUF}(S))} = \varepsilon(n).$$

In Appendix B.1, we formally present the CQ property and prove that a PUF family satisfying the preimage entropy property also satisfies the CQ property. Furthermore, in Appendix B.2, we formally present the indistinguishability property and prove that it follows from the CQ property.

The next property we need is what we call the **challenge-response pair (CRP) guessing** property, which informally states that it should be hard to generate a valid CRP of a PUF, without querying the PUF “close” to the corresponding challenge.¹⁰ This property was used implicitly in the extractability proof in [12] and it is

⁹ Notice that the size of these neighborhoods does not depend on x . More specifically, $|B_n^d| = \sum_{k=0}^{d-1} \binom{n}{k}$.

¹⁰ More precisely, this refers to a tuple (s, st, p) , derived from an actual CRP (s, σ) using a FE.

similarly crucial in our proofs. Although we would ultimately like to reduce it to some of the other PUF properties, we opted to consider it as an additional assumption. It seems that such reductions would require to use specific details about the FE being used, which we wanted to avoid. A short discussion about these possible reductions, as well as the formal statement of the CRP guessing property can be found in Appendix B.3.

Finally, we also need what we call the **test query (TQ)** property¹¹. Suppose an honest party sends its PUF to an adversary who then returns it. How can the honest party be sure that the PUF it received is indeed the one it originally created? In [12], it was noted that the honest party could query the PUF on a randomly selected challenge (a *test query*) and verify the response upon receiving the PUF back. If the returned PUF passes this test, then, with overwhelming probability, it is the original PUF. However, a formal proof of this property was not provided. Its validity depends on the model of malicious PUFs considered. For example, in [14], where an adversary can construct a malicious PUF that encapsulates a PUF received from a different party, the adversary could create a PUF that differs from the original one only on a specific challenge. Clearly, the TQ property would fail in that scenario, which highlights the need to prove it with caution. In Appendix B.4, we present the formal statement of the TQ property, as well as the proof that it follows from the preimage entropy and CRP guessing properties.

To summarize:

- The **response consistency** property is identical to the one used in [10];
- We replace the **extraction independence** property from [10] with a more general **almost-uniformity** property;
- We strengthen the **well-spread domain** assumption from [10] into the **CQ** property, which we derive by additionally assuming the **preimage entropy** property;
- We prove the **indistinguishability** property from the CQ property;
- We assume the **CRP guessing** property as an additional requirement;
- We prove the **TQ** property based on the preimage entropy and CRP guessing properties.

3.2 Fixing the issues in CPUF

The CPUF protocol from [11] is depicted in Fig. 16 in Appendix A. The original security proof provided in [11] is flawed. Specifically, the hiding argument does not adequately address the possibility that an adversary may learn information about s after receiving values that depend on $\text{PUF}(s)$ – precisely where the previously discussed CQ property becomes essential. In addition, the binding proof appears to be incomplete.

To remedy these issues, we adapt the original proof technique from [19]. Our revised hiding argument is based on the indistinguishability property of PUFs, while our binding argument extends the analysis to accommodate the inherent noise of PUFs by leveraging certain entropy guarantees that we prove in Appendix C. Our proof of the following theorem is presented in Appendix E:

Theorem 1. *CPUF is an ideal commitment scheme in the $\mathcal{F}_{\text{MPUF}}$ -hybrid model with unbounded k_{state} .*

4 The ideal extractable commitment from [12]

In [12], the ideal extractable commitment¹² **ExtPUF** (depicted in Fig. 17 in Appendix A) was constructed from the ideal commitment CPUF from [11]. However, we identified a critical flaw in this protocol: the corresponding extractor does not work, and we demonstrate this with an explicit attack. This reveals a deeper limitation in the original design and raises the question of whether extractability can be achieved without imposing stronger restrictions on adversarial behavior. We suggest possible directions for addressing this problem and explore several potential fixes.

4.1 Attack on the ideal extractable commitment from [12]

The most critical issue we identified in [12] concerns the **ExtPUF** protocol and its associated extractor, which are described in Appendix A (see Figs. 17 and 18). We describe an attack on this construction.

Consider a malicious sender S^* that behaves just like an honest S committing to the bit 0, except that it also queries PUF_E on $\text{Enc}(st_1 \oplus r_1)$. Then, $c_1 = st_1$ and $\mathcal{Q} = \{\text{Enc}(st_1), \text{Enc}(st_1 \oplus r_1)\}$, which means

- for $q = \text{Enc}(st_1)$, we have $\text{Dec}(q) \oplus (0^l \wedge r_1) = st_1 = c_1$ and so 0 is extracted;

¹¹ We include this property here for clarity, even though it is not required for the CPUF protocol. It becomes essential later for the ideal extractable protocol **ExtPUF**.

¹² See Definition 12 in Appendix A.

- for $q = \text{Enc}(st_1 \oplus r_1)$, we have $\text{Dec}(q) \oplus (1^l \wedge r_1) = st_1 \oplus r_1 \oplus r_1 = c_1$ and so 1 is extracted.

Therefore, in this case E always outputs $\perp$. However, S^* can always decommit successfully to 0, breaking the extraction property.

The original goal in [12] behind **ExtPUF** was to base its extractability on the binding property of **CPUF**. Indeed, it was meant to force S^* to query PUF_E on an opening of the commitment $\text{CPUF}(x)$, and thus binding it to x . However, as we have seen, st_1 is not an opening of that commitment.

4.2 First solution: notifying PUFs

To prevent the attack described in Section 4.1, we propose a modified version of **ExtPUF** where S sends c_2 before receiving r_1 . The motivation and details of this modification are discussed in Appendix D. However, as discussed in there, this change does not fully resolve the issue.

In this section, we extend the protocol described in Appendix D to a setting where honest PUFs notify their creators upon being queried, and adversaries cannot suppress these notifications, thus proposing the **NotifExtPUF** protocol (see Fig. 5). The corresponding ideal functionality, $\mathcal{F}_{\text{MPUFNotif}}$, is depicted in Fig. 4; the notions from Appendix A can be defined similarly for this model.

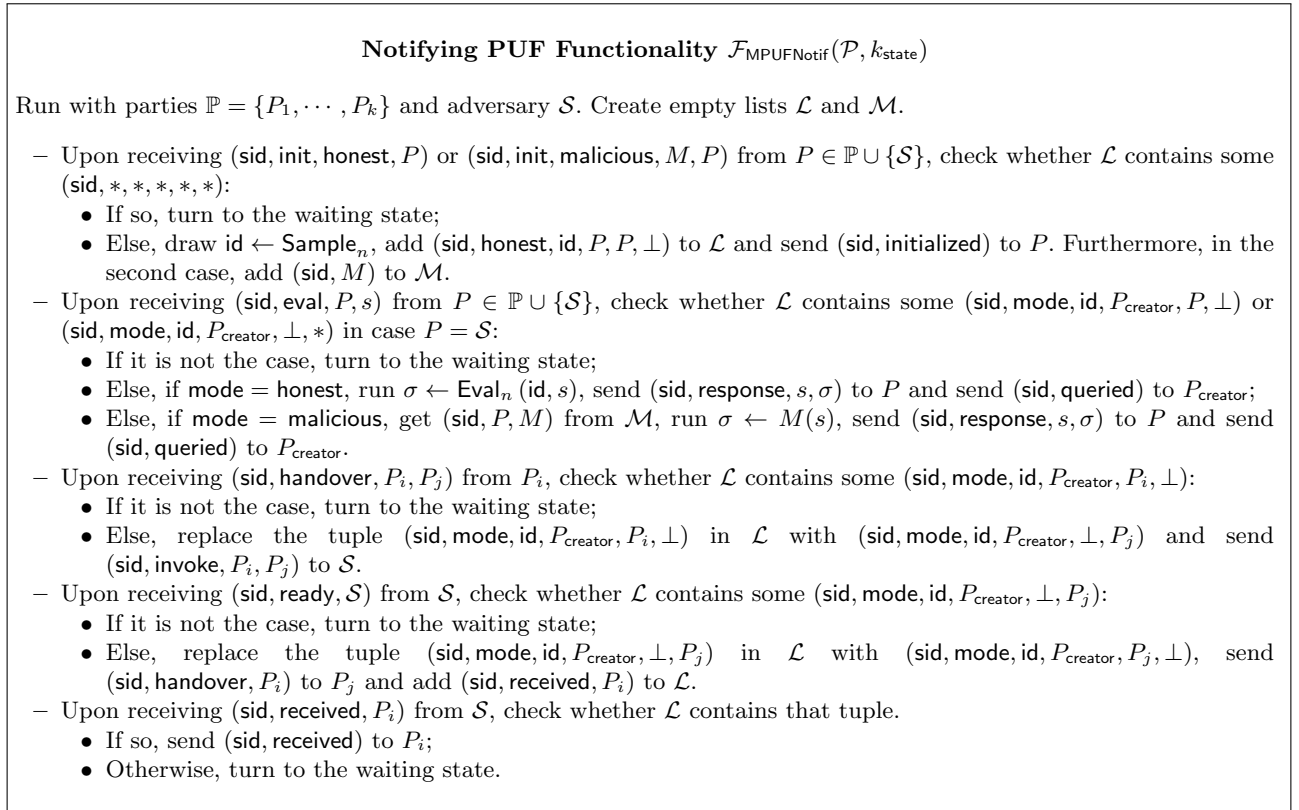


Fig. 4. The notifying PUF functionality.

This model enables us to address the remaining problem: allowing the receiver R to detect whether the malicious sender S^* has queried PUF_E after receiving r_1 . Furthermore – and this applies to any modification of **ExtPUF** that allows R to detect whether S^* has queried PUF_E after receiving r_1 – notice that the sender no longer needs to commit to $st_E \parallel p_E$. The original purpose of this commitment was to force S^* to query PUF_E during the commitment phase, ensuring that the extractor could observe the query. This is no longer necessary, since R can now reject the commitment if S^* performs any queries after the allowed point in the protocol. In addition, the length of the string r can be reduced from $3kn$ to kn , as the protocol no longer relies on the statistical binding of **CPUF**. Recall that in **CPUF**, a malicious sender could use the decommitment phase to leak information about r to PUF by carefully choosing s , which is what necessitated a longer r . In our setting, however, the sender must query PUF_E

with the response from PUF *before* learning r , eliminating this concern. Finally, the protocol can be simplified by reducing the number of PUF exchange phases (i.e., points at which the parties exchange PUFs). This is achieved by having both PUF and PUF_E sent simultaneously. This simplification is a first step toward designing a more efficient UC-secure protocol, which we further explore in Section 6.1.

This protocol is parameterized by the following components:

- a PUF family $\mathcal{P}_E$ and a fuzzy extractor family $\text{FE}_E = (\text{Gen}_E, \text{Rep}_E)$ with matching parameters;
- a family (Enc, Dec) of $(kn, L, 2(d_{\min})_E - 1)$ -error-correcting codes for some L ;
- a PUF family $\mathcal{P}$ and a fuzzy extractor family $\text{FE} = (\text{Gen}, \text{Rep})$ with matching parameters such that $\ell_{\text{FE}}(n) = kn$.

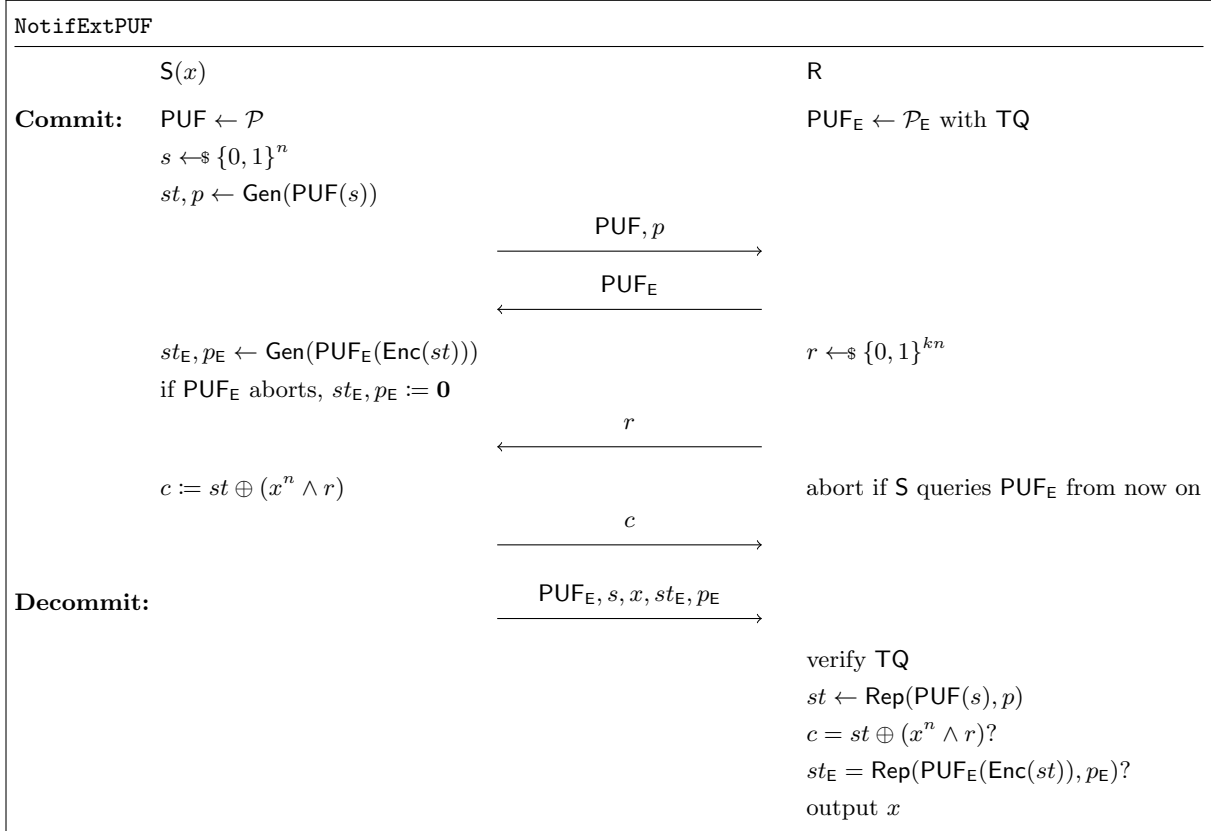


Fig. 5. The proposed NotifExtPUF protocol.

We prove the following theorem in Appendix E:

Theorem 2. *NotifExtPUF is an ideal extractable commitment scheme in the $\mathcal{F}_{\text{MPUFNotif}}$ -hybrid model.*

4.3 Second solution: stateless malicious PUFs

Another way to ensure R is able to detect whether S^* has queried PUF_E after receiving r_1 is to simply change the protocol so that the sender returns PUF_E before receiving r_1 . This proposed **StatelessExtPUF** protocol is depicted in Fig. 6.

Enforcing this, however, introduces a significant limitation: we must assume that malicious PUFs are stateless. Indeed, this is necessary to ensure hiding. If PUF_E retains any information about its prior interactions (such as the challenge st_1), then R^* could recover part of st_1 . Since knowing just one bit of st_1 would allow R^* to break hiding, we need to restrict the model to stateless malicious PUFs. Again, exactly the same simplifications as the ones discussed in Section 4.2 can be made, and so the protocol parameters are exactly the same. We prove the following theorem in Appendix E:

Theorem 3. *StatelessExtPUF is an ideal extractable commitment scheme in the $\mathcal{F}_{\text{MPUF}}$ -hybrid model with $k_{\text{state}} = 0$.*

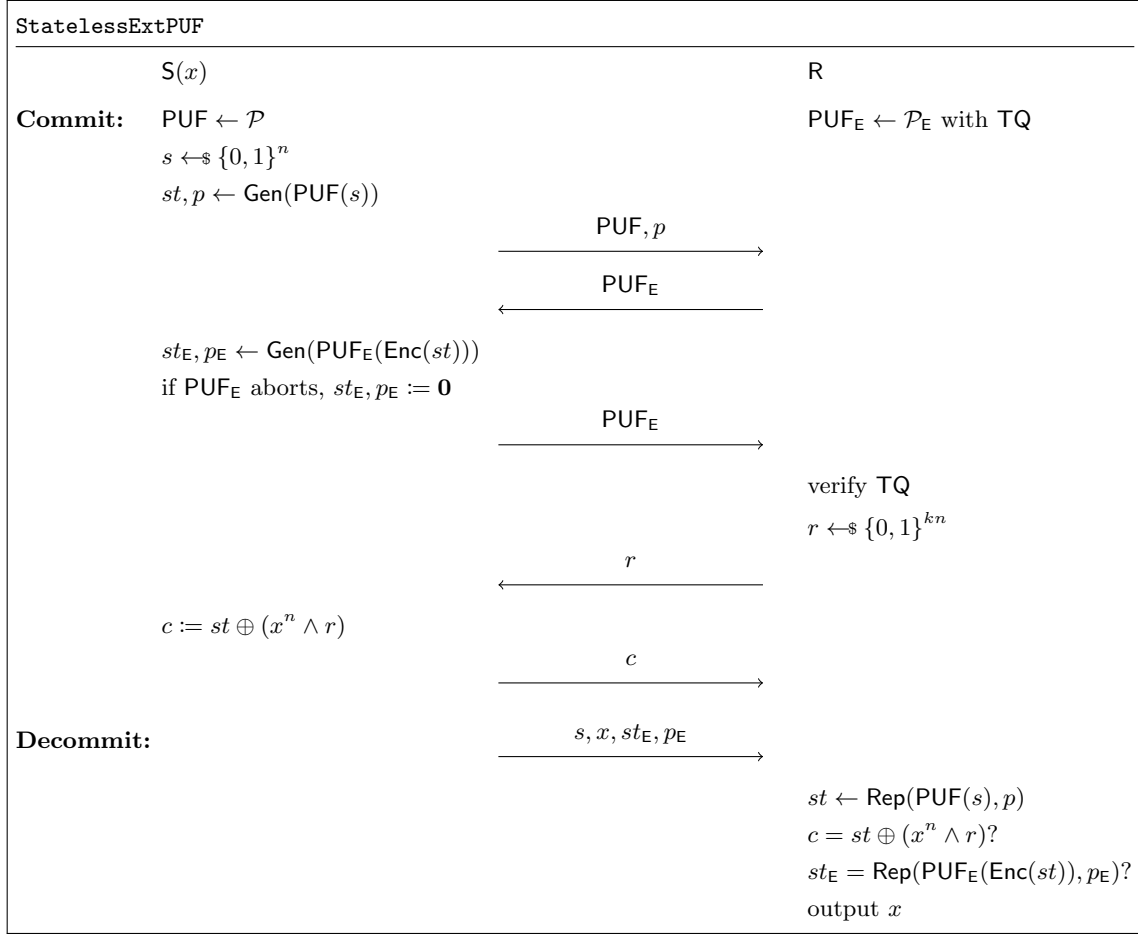


Fig. 6. The proposed StatelessExtPUF protocol.

5 The compiler from [12]

5.1 Collective commitments

As mentioned in Section 1.2, we generalized the definition of commitments to accommodate the commitment of many strings at once. This yields what we call a *collective commitment scheme*, and the corresponding syntax, where $N(n)$ denotes the number of strings being committed and $k(n)$ their size, is given in the following definition. These definitions are made in the context of the $\mathcal{F}_{\text{MPUF}}$ functionality, but they can be analogously defined for the $\mathcal{F}_{\text{MPUFNotif-hybrid}}$ model as well.

Definition 5. A **collective commitment scheme in the $\mathcal{F}_{\text{MPUF}}$ -hybrid model** is a tuple of PPT algorithms $\text{CollCom} = (\text{S}, \text{R})$ that run on security parameter n and have oracle access to $\mathcal{F}_{\text{MPUF}}$, implementing the following functionality:

- **Inputs:** S receives as inputs strings $x^1, \dots, x^{N(n)} \in \{0, 1\}^{k(n)}$.
- **Commitment phase:** S commits to $\mathbf{x} = (x^1, \dots, x^{N(n)})$, which we denote by $\text{Commit}^{\text{CollCom}}(\mathbf{x})$.
- **Decommitment of commitments in a set $I \subseteq [N] = \{1, \dots, N\}$:**
S sends $\{(i, x^i)\}_{i \in I}$ and some decommitment data to R, which outputs either $\{(i, x^i)\}_{i \in I}$, if it accepts the decommitment, or $\perp$, otherwise. We denote this by $\text{Open}^{\text{CollCom}}((x^i)_{i \in I})$, and we refer to this phase as the decommitment of I .

When using this protocol, there can be many decommitment phases, not necessarily at the same time.

In the following, consider a fixed collective commitment scheme in the $\mathcal{F}_{\text{MPUF}}$ -hybrid model, where we only consider functions $N(n)$ and $k(n)$ that are polynomial in n . We need to define what it means for such a protocol

to be hiding. The intuitive idea is that the committed strings remain hidden until they are revealed — even if other strings have already been opened. This property must hold even within a more complex interaction. Of course, this interaction must be restricted in certain ways. For instance, if x is one of the strings committed by the sender S and S later sends x to a malicious receiver R^* , it would no longer remain hidden, even without explicitly opening the commitment. Therefore, S must be restricted from sending any messages that depend on the strings intended to remain hidden throughout the interaction. Additionally, S must be restricted from sharing any information generated during the commitment phase, as this could aid R^* in learning about the committed strings. Thus, S should only interact with the commitment scheme as a black box, ensuring no internal details are leaked. Furthermore, the interaction may involve S not knowing which strings it will commit to at the start. However, there must be a clear point where S defines the strings and decides which ones will eventually be revealed. We formalize this in the following definition:

Definition 6. Consider the interaction between an honest sender S and a malicious receiver R^* depicted in Fig. 7, where $\{\Omega_n\}_{n \in \mathbb{N}}$ is a collection of finite sets and $\text{INTER}^{\text{CollCom}}$ denotes an interaction such that:

- S and R^* can interact arbitrarily, as long as the messages sent by S do not depend on w ;
- There is a moment in the interaction where S defines a function $\text{STRINGS}_n : \Omega_n \rightarrow (\{0, 1\}^k)^{N(n)}$ and sets $\text{OPEN}_n \subseteq \text{Const}(\text{STRINGS}_n)$ ¹³ and $\text{CLOSED}_n := [N] \setminus \text{OPEN}_n$;
- After that, S commits to $\text{STRINGS}_n(w)$ using the protocol CollCom as a black-box. This is the only time in the interaction where S has access to w . Furthermore, since S runs the commitment as a black-box, it does not have access to the information of the commitment outside the commitment and decommitment interactions.¹⁴
- S decommits the strings in OPEN_n (not necessarily at the same time).

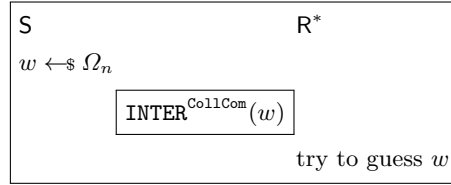


Fig. 7. Collective hiding interaction.

We say that CollCom is **computationally hiding** if all interactions INTER and PPT malicious receivers R^* in the interaction depicted in Fig. 7 satisfy

$$\Pr[R^*(\text{INTER}^{\text{CollCom}}(W)) = W] = \frac{1}{|\Omega_n|} + \varepsilon(n).$$

Following the same idea, we also need to define the binding property within a more complex interaction. As with the hiding definition, we ensure that R interacts with the commitment scheme as a black box, preventing it from sending any information that could aid the malicious sender S^* .

Definition 7. Consider the interaction between a malicious sender S^* and an honest receiver R depicted in Fig. 8, where INTER denotes an interaction where S^* makes a possibly malicious commitment using CollCom . Throughout this interaction, R uses CollCom honestly as a black-box. Furthermore, d_1 and d_2 denote two sequences of actions that lead to decommitments.¹⁵ We say that S^* is successful when d_1 and d_2 lead to successful decommitments of some $i \in [N]$ to different strings.

We say that CollCom is **statistically binding** if for all interactions INTER , all malicious senders S^* succeed with negligible probability.

Finally, the same happens for extractability:

Definition 8. Consider the interaction between a malicious sender S^* and some extractor E depicted in Fig. 9, where INTER denotes an interaction in which S^* makes a possibly malicious commitment using CollCom . We say that CollCom is **extractable** if there exists a PPT extractor E having interface access to $\mathcal{F}_{\text{MPUF}}$ such that all interactions INTER and malicious committers S^* satisfy the following properties:

¹³ For a function $f = (f^1, \dots, f^N) : X \rightarrow Y^N$, we define $\text{Const}(f) = \{i \in [N] : f^i \text{ is constant}\}$.

¹⁴ Of course the same cannot be said about R^* , since it is malicious.

¹⁵ This can include some decommitments.

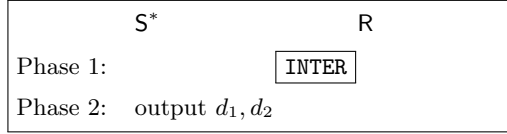


Fig. 8. Collective binding interaction.

- **Simulation:** *The view of S^* when interacting with E is identical to the view when interacting with an honest receiver R .*
- **Extraction:** *S^* only decommits successfully to some string that is different from what E outputs with negligible probability, that is,*

$$\Pr\left[S^* \text{ decommits some } i \text{ successfully to } X^i \neq (X^*)^i\right] = \varepsilon(n).$$

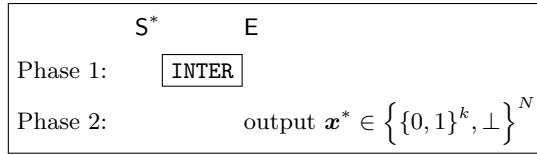


Fig. 9. Collective extractability interaction.

Just as before, we can define the concepts of ideal and ideal extractable collective commitment schemes in the $\mathcal{F}_{\text{MPUF}}$ -hybrid model analogously.

5.2 Revising the compiler from [12]

As previously discussed, since we are working with multiple commitments, we need to use a collective commitment scheme **CollCom**. This approach not only ensures that the security of these commitments is maintained, but also allows us to optimize the protocol. Along these lines, we need to revise accordingly the compiler from [12], obtaining the **UCCompiler** depicted in Fig. 10¹⁶. The latter uses a parallelized version of the **BlobEquality** protocol from [12], which we call **BlobEqualities** and it is depicted in Fig. 11. We thus prove the following result in Appendix F:

Theorem 4. *Let **CollCom** be an ideal extractable collective commitment scheme in the $\mathcal{F}_{\text{MPUF}}$ -hybrid model. Then, the corresponding protocol given by **UCCompiler** UC-realizes $\mathcal{F}_{\text{com}}$ in the $\mathcal{F}_{\text{MPUF}}$ -hybrid model.*

The main difference in our construction is that, unlike in **BlobEquality** from [12], R 's commitment to $\mathbf{e}$ is made earlier in the **UCCompiler** protocol, even before S 's commitment; and it is exactly this early commitment which ensures that the UC proof holds.

6 Construction of UC-secure commitments

According to the two solutions we proposed in Sections 4.2 and 4.3 in order to make the construction resist our attack presented in Section 4.1, here we propose the respective two collective commitment protocols, detailed in Appendix E:

- **CollNotifExtPUF**, a collective version of **NotifExtPUF**, which operates in the $\mathcal{F}_{\text{MPUFNotif}}$ -hybrid model;
- **CollStatelessExtPUF**, a collective version of **StatelessExtPUF**, which works in the $\mathcal{F}_{\text{MPUF}}$ -hybrid model assuming stateless PUFs.

¹⁶ Therein, we define $[n, 2] = \{1, 3, \dots, 2n + 1\}$.

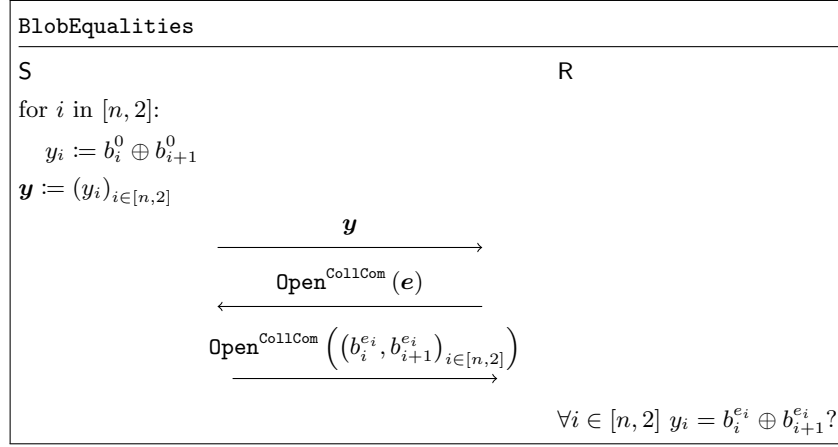


Fig. 11. The **BlobEqualities** protocol, which is a parallelized version of **BlobEquality**.

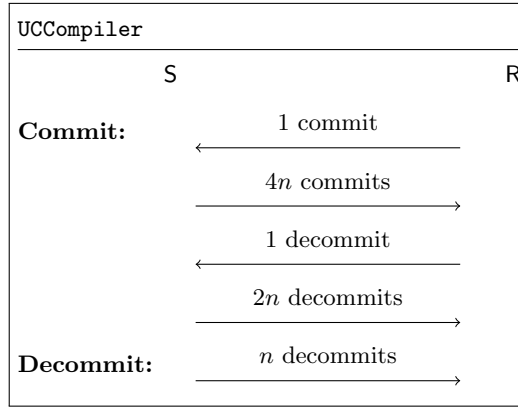


Fig. 12. A schematic representation of the commitment and decommitment phases in **UCCompiler**.

phases, while no exchanges are needed during decommitment. Therefore, the resulting UC-secure commitment would require a total of

$$2(4n + 1) = 8n + 2$$

PUFs and an equal number of PUF exchange phases.

Now, let us look at the efficiency when using our collective commitment protocols. In both cases, we employ two collective commitments – one for each direction.

The **CollNotifExtPUF** protocol requires the sender to create one PUF and the receiver to create N PUFs, where N is the number of strings being committed, resulting in a total of $N + 1$ PUFs. So, the resulting UC-secure commitment would require a total of

$$(1 + 1) + (4n + 1) = 4n + 3$$

PUFs. Each collective commitment and decommitment phase involves one PUF exchange, totaling five PUF exchange phases.

On the other hand, the **CollStatelessExtPUF** protocol requires the creation of one PUF by each party; the resulting UC-secure commitment would then require a total of four PUFs. Furthermore, two PUF exchange phases are needed in the commitment phase, while no exchanges are required in the decommitment phases. Thus, this results in a total of four PUF exchange phases.

To summarize:

- The **CollNotifExtPUF** protocol offers a significant efficiency improvement over naive instantiations of **ExtPUF** by cutting the number of PUFs nearly in half and limiting the number of exchange phases to a small constant;
- The **CollStatelessExtPUF** protocol achieves the best efficiency, requiring only 4 PUFs and 4 PUF exchanges overall.

7 Acknowledgements

This work is funded by Fundação para a Ciência e a Tecnologia (FCT), Portugal FCT/MECI through national funds and when applicable co-funded EU funds under the Unit UIDB/50008 and the project PUFSeQure (2023.14154.PEX). CV also acknowledges support from the FCT under the Scientific Employment Stimulus - Individual Call (CEEC Individual) - 2020.03274.CEECIND/CP1621/CT000.

References

1. R. Canetti, “Security and Composition of Multiparty Cryptographic Protocols,” *J. Cryptology*, vol. 13, pp. 143–202, 2000. <https://doi.org/10.1007/s001459910006>
2. R. Canetti, “Universally composable security: A new paradigm for cryptographic protocols,” In *Proceedings 42nd IEEE Symposium on Foundations of Computer Science*, pp. 136–145, 2001. <https://doi.org/10.1109/SFCS.2001.959888>
3. R. Canetti, and M. Fischlin, “Universally composable commitments,” In Kilian, J. (eds) *Advances in Cryptology - CRYPTO 2001. Lecture Notes in Computer Science*, vol. 2139, pp. 19–40, Springer, Berlin, Heidelberg, 2001. https://doi.org/10.1007/3-540-44647-8_2
4. R. Canetti, Y. Lindell, R. Ostrovsky, and A. Sahai, “Universally composable two-party and multi-party secure computation,” In *Proceedings of the 34th annual ACM Symposium on Theory of Computing*, pp. 494–503, 2002. <https://doi.org/10.1145/509907.509980>
5. R. Canetti, E. Kushilevitz, and Y. Lindell, “On the Limitations of Universally Composable Two-Party Computation without Set-up Assumptions,” In Biham, E. (eds) *Advances in Cryptology — EUROCRYPT 2003. Lecture Notes in Computer Science*, vol. 2656, pp. 68–86, Springer, Berlin, Heidelberg, 2003. https://doi.org/10.1007/3-540-39200-9_5
6. R. Canetti, Y. Dodis, R. Pass, and S. Walfish, “Universally Composable Security with Global Setup,” In Vadhan, S.P. (eds) *Theory of Cryptography - TCC 2007. Lecture Notes in Computer Science*, vol. 4392, pp. 61–85, Springer, Berlin, Heidelberg, 2007. https://doi.org/10.1007/978-3-540-70936-7_4
7. Y. Ishai, M. Prabhakaran, and A. Sahai, “Founding cryptography on oblivious transfer—efficiently,” In Wagner, D. (eds) *Advances in Cryptology – CRYPTO 2008. Lecture Notes in Computer Science*, vol. 5157, pp. 572–591, Springer, Berlin, Heidelberg, 2008. https://doi.org/10.1007/978-3-540-85174-5_32
8. R. Canetti, “Universally composable security,” *J. ACM*, vol. 67, pp. 1–94, 2020. <https://doi.org/10.1145/3402457>
9. R. Pappu, B. Recht, J. Taylor, and N. Gershenfeld, “Physical One-Way Functions,” *Science*, vol. 297, pp. 2026–2030, 2002. <https://www.science.org/doi/abs/10.1126/science.1074376>
10. C. Brzuska, M. Fischlin, H. Schröder, and S. Katzenbeisser, “Physically uncloneable functions in the universal composition framework,” In Rogaway, P. (eds) *Advances in Cryptology – CRYPTO 2011. Lecture Notes in Computer Science*, vol. 6841, pp. 51–70, Springer, Berlin, Heidelberg, 2011. https://doi.org/10.1007/978-3-642-22792-9_4
11. R. Ostrovsky, A. Scafuro, I. Visconti, and A. Wadia, “Universally composable secure computation with (malicious) physically uncloneable functions,” In Johansson, T., Nguyen, P.Q. (eds) *Advances in Cryptology – EUROCRYPT 2013. Lecture Notes in Computer Science*, vol. 7881, pp. 702–718, Springer, Berlin, Heidelberg, 2013. https://doi.org/10.1007/978-3-642-38348-9_41
12. I. Damgård, and A. Scafuro, “Unconditionally secure and universally composable commitments from physical assumptions,” In Sako, K., Sarkar, P. (eds) *Advances in Cryptology - ASIACRYPT 2013. Lecture Notes in Computer Science*, vol. 8270, pp. 100–119, Springer, Berlin, Heidelberg, 2013. https://doi.org/10.1007/978-3-642-42045-0_6
13. D. Dachman-Soled, N. Fleischhacker, J. Katz, A. Lysyanskaya, and D. Schröder, “Feasibility and infeasibility of secure computation with malicious PUFs,” In Garay, J.A., Gennaro, R. (eds) *Advances in Cryptology – CRYPTO 2014. Lecture Notes in Computer Science*, vol. 8617, pp. 405–420, Springer, Berlin, Heidelberg, 2014. https://doi.org/10.1007/978-3-662-44381-1_23
14. S. Badrinarayanan, D. Khurana, R. Ostrovsky, and I. Visconti, “Unconditional UC-secure computation with (stronger-malicious) PUFs,” In Coron, J.S., Nielsen, J. (eds) *Advances in Cryptology – EUROCRYPT 2017. Lecture Notes in Computer Science*, vol. 10210, pp. 382–411, Springer, Cham 2017. https://doi.org/10.1007/978-3-319-56620-7_14
15. B. Magri, G. Malavolta, D. Schröder, and D. Unruh, “Everlasting UC Commitments from Fully Malicious PUFs,” *J. Cryptology*, vol. 35, 20, 2022. <https://doi.org/10.1007/s00145-022-09432-4>
16. R. Maes, “Physically unclonable functions: Constructions, properties and applications,” Springer Berlin, Heidelberg, 2013. <https://doi.org/10.1007/978-3-642-41395-7>
17. J. Katz, “Universally Composable Multi-party Computation Using Tamper-Proof Hardware,” In Naor, M. (eds) *Advances in Cryptology - EUROCRYPT 2007. Lecture Notes in Computer Science*, vol. 4515, pp. 115–128, Springer, Berlin, Heidelberg, 2007. https://doi.org/10.1007/978-3-540-72540-4_7
18. D. Unruh, “Universally composable quantum multi-party computation,” In Gilbert, H. (eds) *Advances in Cryptology – EUROCRYPT 2010. Lecture Notes in Computer Science*, vol. 6110, pp. 486–505, Springer, Berlin, Heidelberg, 2010. https://doi.org/10.1007/978-3-642-13190-5_25
19. M. Naor, “Bit commitment using pseudorandomness,” *J. Cryptology*, vol. 4, pp. 151–158, 1991. <https://doi.org/10.1007/BF00196774>

20. Y. Dodis, L. Reyzin, and A. Smith, “Fuzzy extractors: How to generate strong keys from biometrics and other noisy data,” In Cachin, C., Camenisch, J.L. (eds) *Advances in Cryptology - EUROCRYPT 2004*. Lecture Notes in Computer Science, vol. 3027, pp. 523–540, Springer, Berlin, Heidelberg, 2004. https://doi.org/10.1007/978-3-540-24676-3_31
21. M. Schultz-Wu, “Is this probability negligible?”, in <https://crypto.stackexchange.com/questions/109679/is-this-probability-negligible>, 2024.

A Definitions from [12]

Definition 9. A commitment scheme in the $\mathcal{F}_{\text{MPUF}}$ -hybrid model is a tuple of PPT algorithms $\text{Com} = (S, R)$ that run on security parameter n and have oracle access to $\mathcal{F}_{\text{MPUF}}$, implementing the following functionality:

- **Inputs:** S receives as input a string $x \in \{0, 1\}^k$.
- **Commitment phase:** S interacts with R to commit to the string x ; we denote this by $\text{Commit}^{\text{Com}}(x)$.
- **Decommitment phase:** S sends x and some decommitment data to R , which outputs either x , if it accepts the decommitment, or $\perp$, otherwise; we denote this by $\text{Open}^{\text{Com}}(x)$.

Notice that, in the definition above, we allow commitments to strings of any length k , rather than restricting them to bit commitments as was done in [12].

Definition 10. A commitment scheme $\text{Com} = (S, R)$ is an **ideal commitment scheme in the $\mathcal{F}_{\text{MPUF}}$ -hybrid model** if it satisfies the following properties:

- **Completeness:** If S and R follow their prescribed strategy, then R accepts the decommitment with probability 1.
- **Computationally Hiding:** Let x^0 and x^1 be different strings in $\{0, 1\}^k$. Consider the interaction between an honest sender S and a malicious receiver R^* depicted in Fig. 13¹⁷.

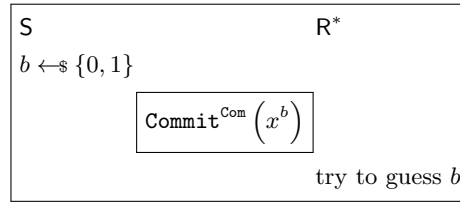


Fig. 13. Hiding interaction.

We say that Com is **computationally hiding** if for all PPT malicious receivers R^* ,

$$\Pr[R^*(\text{Commit}^{\text{Com}}(x^B)) = B] = \frac{1}{2} + \varepsilon(n).$$

- **Statistically Binding:** Consider the interaction between a malicious sender S^* and an honest receiver R depicted in Fig. 14, where Commit denotes the commitment phase of Com , in which S^* may behave in a malicious way. Furthermore, d_1 and d_2 denote two sequences of actions that lead to decommitments. We say that S^* is successful when d_1 and d_2 lead to successful decommitments to different strings.

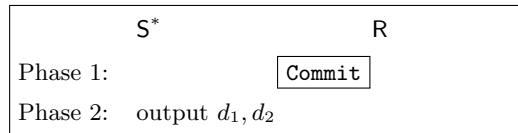


Fig. 14. Binding interaction.

We say that Com is **statistically binding** if all malicious senders S^* in the interaction depicted in Fig. 14 succeed with negligible probability.

¹⁷ Here, S acts as a challenger and R^* as a distinguisher.

In the definition above, we adopted a different (but equivalent) formulation for hiding, where a malicious receiver R^* acts as a distinguisher attempting to guess the committed string. We found that this provides a clearer framework for proving our results.

The protocols constructed in [12] were described as ideal, meaning both statistical hiding and statistical binding. However, they were only statistically hiding under the assumption that the adversary makes a polynomial number of queries to $\mathcal{F}_{\text{MPUF}}$. Rather than relying on this assumption, we instead restrict our analysis to PPT adversaries, who are inherently limited to making a polynomial number of queries. However, for simplicity, we continue to refer to them as ideal.

Definition 11. An algorithm M has **interface access to the functionality** $\mathcal{F}_{\text{MPUF}}$ with respect to a protocol in the $\mathcal{F}_{\text{MPUF}}$ -hybrid model if M has oracle access to $\mathcal{F}_{\text{MPUF}}$ and can observe any query made by any party to honest PUFs during the protocol execution.

Definition 12. A commitment scheme $\text{Com} = (S, R)$ is an **ideal extractable commitment scheme in the $\mathcal{F}_{\text{MPUF}}$ -hybrid model** if Com is an ideal commitment and there exists a PPT extractor E having interface access to $\mathcal{F}_{\text{MPUF}}$ such that, for all malicious senders S^* , it interacts with S^* as depicted in Fig. 15 and satisfies the following properties:

- **Simulation:** The view of S^{*18} when interacting with E is identical to the view when interacting with an honest receiver R .
- **Extraction:** S^* only decommits successfully to some string that is different from what E outputs¹⁹ with negligible probability, that is,

$$\Pr[S^* \text{ decommits successfully to } X \neq X^*] = \varepsilon(n).$$

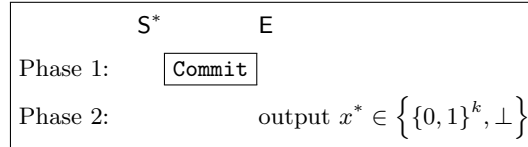


Fig. 15. Extractability interaction.

We will also need the definition below, adapted from [12]:

Definition 13. An (N, L, D) -**error-correcting code (ECC)** is a tuple of PPT algorithms (Enc, Dec) , where $\text{Enc} : \{0, 1\}^N \rightarrow \{0, 1\}^L$ and $\text{Dec} : \{0, 1\}^L \rightarrow \{0, 1\}^N$, such that:

- **Minimum distance:** For all messages $m_1, m_2 \in \{0, 1\}^N$, the corresponding codewords are such that $d(\text{Enc}(m_1), \text{Enc}(m_2)) \geq D^{20}$.
- **Correct decoding:** Let $m \in \{0, 1\}^N$ and $c = \text{Enc}(m)$. Then, for all $c' \in \{0, 1\}^L$,

$$d(c, c') \leq \left\lfloor \frac{D-1}{2} \right\rfloor \implies \text{Dec}(c') = m.$$

The ideal commitment scheme CPUF from [11] is depicted in Fig. 16. We consider a PUF family $\mathcal{P}$ and a fuzzy extractor family $\text{FE} = (\text{Gen}, \text{Rep})$ with matching parameters such that $\ell_{\text{FE}}(n) = kl$, with $l(n) = 3n$.

The ideal extractable commitment ExtPUF proposed in [12] is depicted in Fig. 17 and the corresponding extractor in Fig. 18. This protocol is parameterized by the following components:

- a PUF family $\mathcal{P}_E$ and a fuzzy extractor family $\text{FE}_E = (\text{Gen}_E, \text{Rep}_E)^{21}$ with matching parameters;

¹⁸ That is, the distribution of the messages exchanged during the interaction, as well as S^* 's private information.

¹⁹ Notice that if E outputs $\perp$, this means that S^* cannot decommit successfully to any string.

²⁰ Consequently, D is called the *minimum distance* of the code.

²¹ For simplicity, however, in the protocol description we omit the notation specific fuzzy extractors and write Gen and Rep generically, assuming the appropriate fuzzy extractor is used with each PUF. This convention will be followed in subsequent protocols as well.

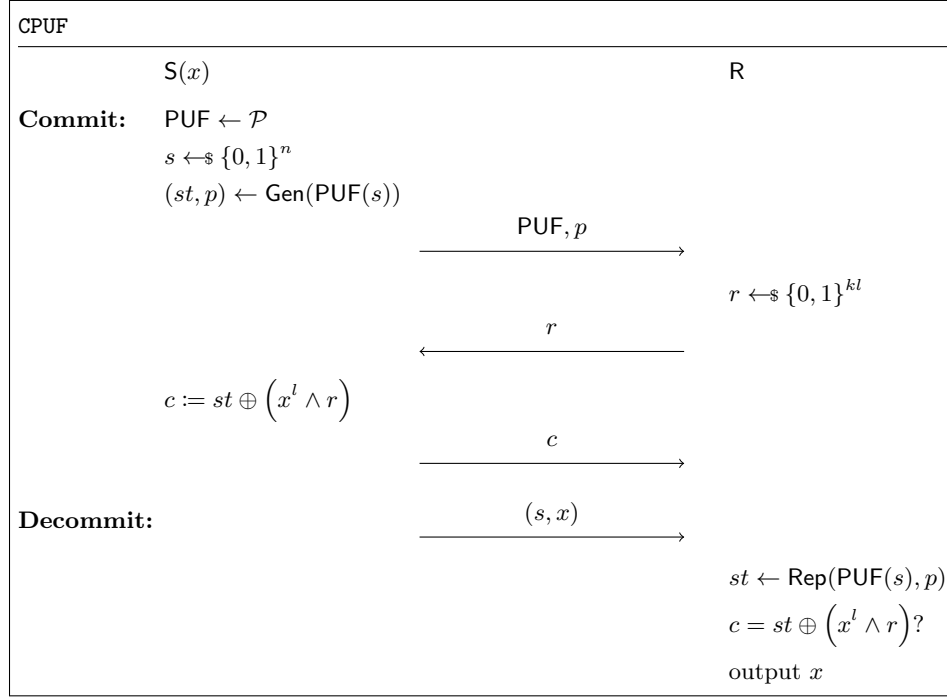


Fig. 16. The ideal commitment scheme CPUF in the $\mathcal{F}_{\text{MPUF}}$ -hybrid model.

- a family (Enc, Dec) of $(kl, L, 2(d_{\min})_{\mathbb{E}} - 1)$ -error-correcting codes²² for some L , with $l(n) = 3n$;
- a PUF family $\mathcal{P}_1$ and a fuzzy extractor family $\text{FE}_1 = (\text{Gen}_1, \text{Rep}_1)$ with matching parameters such that $\ell_{\text{FE}_1}(n) = kl$;
- a PUF family $\mathcal{P}_2$ and a fuzzy extractor family $\text{FE}_2 = (\text{Gen}_2, \text{Rep}_2)$ with matching parameters such that $\ell_{\text{FE}_2}(n) = ml$, with $m = |st_{\mathbb{E}}| \parallel p_{\mathbb{E}}|$.

Furthermore, in the protocol description **TQ** denotes a test query.

B PUF properties

B.1 CQ property

We formalize the CQ property as an interaction between an honest challenger $\mathcal{C}$ and an adversary $\mathcal{A}$, depicted in Fig. 19, where $\mathcal{A}$ is successful if it can query PUF on a challenge that is close to s . We say that $\mathcal{P}$ satisfies the CQ property if any adversary succeeds with negligible probability.

Proposition 1. *If $\mathcal{P}$ satisfies the preimage entropy property, then it satisfies the CQ property.*

Proof. Let $\mathbf{Q}$ be the sequence of queries $\mathcal{A}$ makes to PUF, which is a random variable. Notice that

$$\Pr[\mathcal{A} \text{ is successful}] = \Pr[d(\mathbf{Q}, S) < d_{\min}(n)] = \Pr[\exists Q \in \mathbf{Q} : d(Q, S) < d_{\min}(n)] \leq \sum_{Q \in \mathbf{Q}} \Pr[d(Q, S) < d_{\min}(n)].$$

Since $\mathcal{A}$ is PPT, it only queries PUF a polynomial number of times and therefore it suffices to show that $\Pr[d(Q, S) < d_{\min}(n)]$ is negligible for each $Q \in \mathbf{Q}$.

First, notice that $Q = f(\text{PUF}(S), X)$, for some function f and some random variable $X \perp\!\!\!\perp S$. Indeed, if Q is the first query, this is clear, since at that point the only information that $\mathcal{A}$ has that depends on S is $\text{PUF}(S)$. Furthermore, suppose some queries Q' were done before Q and all of them are of that form. Now, Q can additionally

²² See Definition 13 in Appendix A.

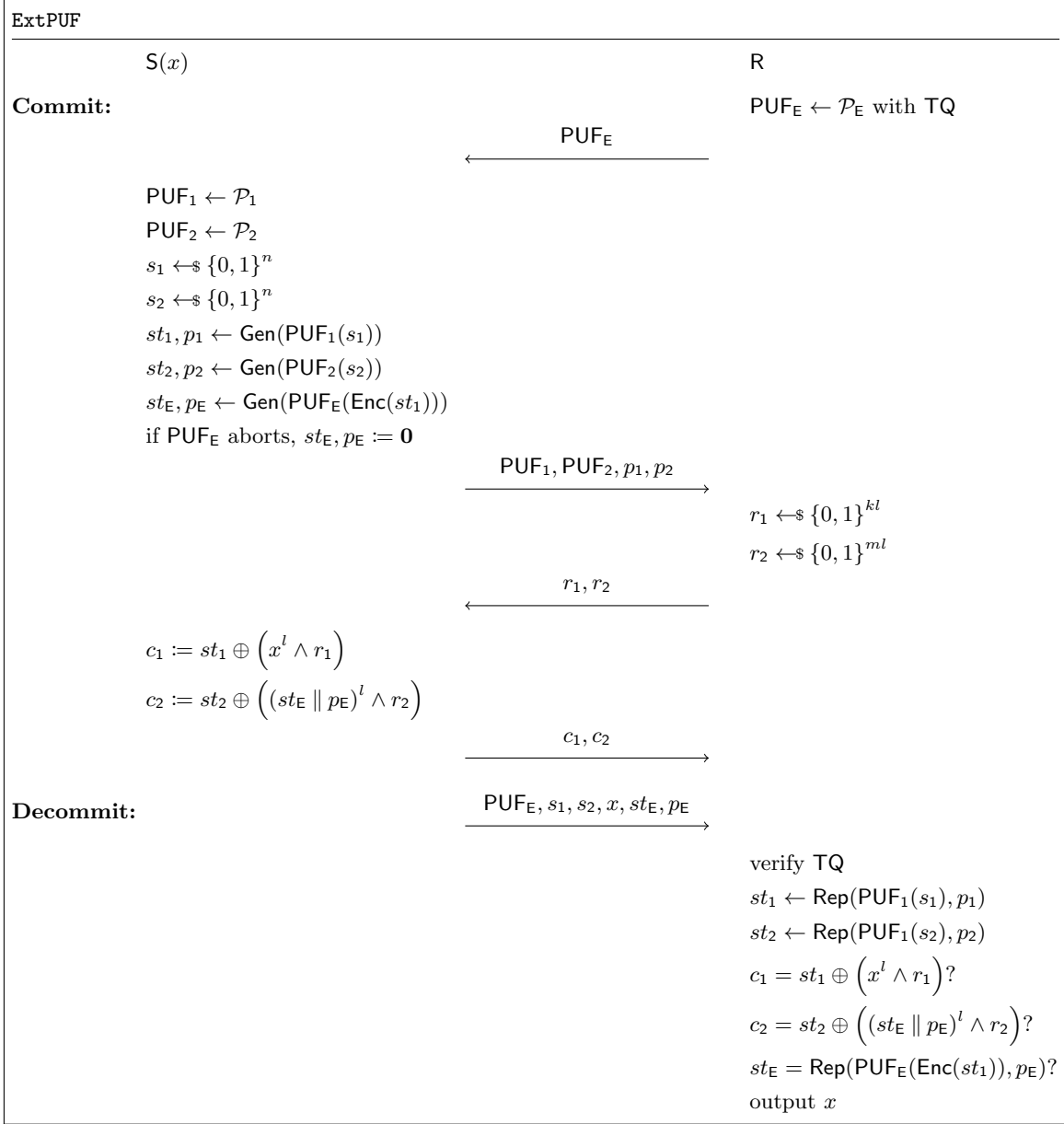


Fig. 17. The original ExtPUF protocol.

Extractor E

E proceeds like an honest receiver R, while also saving S^* 's queries to PUF_E in $\mathcal{Q}$. For each $j \in [k]$, let $I_j = \{j, j+k, \dots, j+(l-1)k\}$. Once the commitment phase is completed, for each $q \in \mathcal{Q}$, it tries to extract a string x from q by running $\text{ExtractFromQuery}(\text{Dec}(q))$. This procedure is defined as follows (here, ε denotes the empty string):

```

ExtractFromQuery( $st$ )
 $x := \varepsilon$ 
for  $j \in [k]$  :
  if  $(c_1)_{I_j} = st_{I_j} \wedge (c_1)_{I_j} \neq st_{I_j} \oplus (r_1)_{I_j}$  :
     $x := x \parallel 0$ 
  elseif  $(c_1)_{I_j} = st_{I_j} \oplus (r_1)_{I_j} \wedge (c_1)_{I_j} \neq st_{I_j}$  :
     $x := x \parallel 1$ 
  else :
    return  $\perp$ 
return  $x$ 

```

Then, it does the following:

- If exactly one string x was extracted, output $x^* = x$;
- Otherwise, output $x^* = \perp$.

Fig. 18. The original extractor defined in [12].

depend on $\text{PUF}(Q')$, but since all the Q' are of that form, the same will happen with Q . From this observation, we get

$$\begin{aligned}
 \Pr[\mathbf{d}(Q, S) < \mathbf{d}_{\min}(n)] &= \Pr[S \in B_n^{\mathbf{d}_{\min}(n)}(Q)] \leq \left| B_n^{\mathbf{d}_{\min}(n)} \right| 2^{-\tilde{H}_{\infty}(S \mid Q)} & (\text{Lemma 9}) \\
 &\leq \left| B_n^{\mathbf{d}_{\min}(n)} \right| 2^{-\tilde{H}_{\infty}(S \mid \text{PUF}(S), X)} & (\text{Lemma 4}) \\
 &\leq \left| B_n^{\mathbf{d}_{\min}(n)} \right| 2^{-\tilde{H}_{\infty}(S \mid \text{PUF}(S))}, & (\text{Lemma 5})
 \end{aligned}$$

which is negligible, by assumption.

Remark 1. In [10], it was assumed that $\mathbf{d}_{\min}(n) \in o(n/\log(n))$, which corresponds to a simplified version of the preimage entropy property. Indeed, first notice that

$$\left| B_n^{\mathbf{d}_{\min}(n)} \right| = \sum_{k=0}^{\mathbf{d}_{\min}(n)-1} \binom{n}{k} \leq \sum_{k=0}^{\mathbf{d}_{\min}(n)-1} n^k = \frac{n^{\mathbf{d}_{\min}(n)} - 1}{n - 1} \leq n^{\mathbf{d}_{\min}(n)}.$$

Since $\tilde{H}_{\infty}(S \mid \text{PUF}(S))$ was not taken into consideration (that is, it was assumed to take its maximum value n), this then leads to

$$\left| B_n^{\mathbf{d}_{\min}(n)} \right| 2^{-\tilde{H}_{\infty}(S \mid \text{PUF}(S))} \leq n^{\mathbf{d}_{\min}(n)} 2^{-n} \leq 2^{-n+\mathbf{d}_{\min}(n) \log(n)} = 2^{-n+o(n)},$$

which is negligible.

B.2 Indistinguishability property

We can formalize the indistinguishability property as an interaction between an honest challenger $\mathcal{C}$ and a distinguisher $\mathcal{D}$, depicted in Fig. 20, where $\mathcal{D}$ is successful if it can guess whether the extracted string it received comes

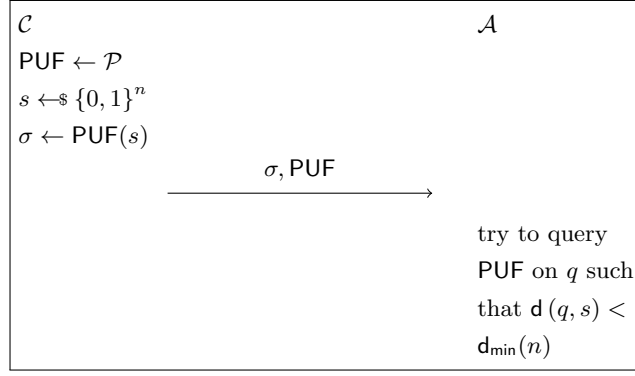


Fig. 19. CQ property interaction.

from PUF or is uniform. We say that $\mathcal{P}$ satisfies the indistinguishability property if all distinguishers succeed with probability negligibly close to $\frac{1}{2}$, that is,

$$\Pr[\mathcal{D} = B] = \frac{1}{2} + \varepsilon(n).$$

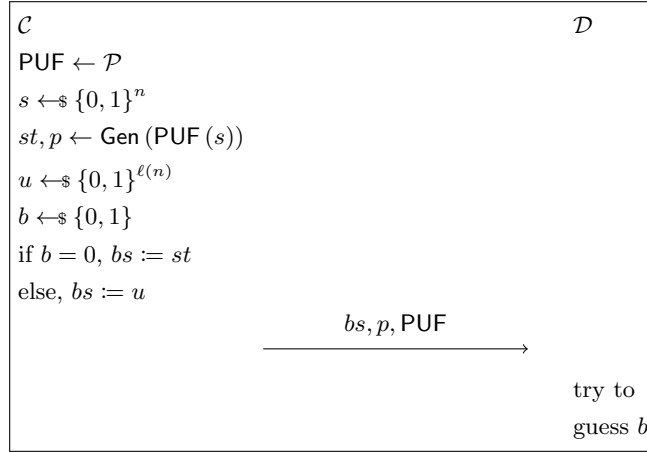


Fig. 20. Indistinguishability property interaction.

Lemma 1. Consider the indistinguishability property interaction depicted in Fig. 20 and let $\mathbf{Q}$ denote the sequence of queries $\mathcal{D}$ makes to PUF. Then, for all distinguishers $\mathcal{D}$,

$$\Pr[\mathcal{D} = B \mid d(\mathbf{Q}, S) \geq d_{\min}(n)] = \frac{1}{2} + \varepsilon(n).$$

Proof. Suppose $\mathbf{Q} = \mathbf{q}$ and $S = s$ such that $d(\mathbf{q}, s) \geq d_{\min}(n)$. Consider the programs depicted in Fig. 21.

We get $\tilde{H}_\infty(\text{PUF}(s) \mid \text{PUF}(\mathbf{q})) \geq m(n)$ from unpredictability. Thus, from almost-uniformity with $\text{EXTRA} = \text{PUF}(\mathbf{q})$, we know that

$$\text{SD}(D_0, D_1) = |\varepsilon(n)|.$$

Furthermore, notice that each D_b corresponds to the information $\mathcal{D}$ gets in the interaction if $B = b$. Therefore, since statistical closeness implies indistinguishability, we get

$$\Pr[\mathcal{D} = B \mid \mathbf{Q} = \mathbf{q}, S = s] = \frac{1}{2} + \varepsilon(n).$$

D_0	D_1
$\text{PUF} \leftarrow \mathcal{P}$	$\text{PUF} \leftarrow \mathcal{P}$
$\sigma \leftarrow \text{PUF}(s)$	$\sigma \leftarrow \text{PUF}(s)$
$(st, p) \leftarrow \text{Gen}(\sigma)$	$(st, p) \leftarrow \text{Gen}(\sigma)$
$\sigma_q \leftarrow \text{PUF}(q)$	$u \leftarrow \{0, 1\}^{\ell(n)}$
return (st, p, σ_q)	$\sigma_q \leftarrow \text{PUF}(q)$
	return (u, p, σ_q)

Fig. 21. Programs D_0 and D_1 .

Thus,

$$\begin{aligned}
\Pr[\mathcal{D} = B, d(\mathbf{Q}, S) \geq d_{\min}(n)] &= \sum_{\substack{(\mathbf{q}, s): \\ d(\mathbf{q}, s) \geq d_{\min}(n)}} \Pr[\mathcal{D} = B \mid \mathbf{Q} = \mathbf{q}, S = s] \Pr[\mathbf{Q} = \mathbf{q}, S = s] \\
&= \sum_{\substack{(\mathbf{q}, s): \\ d(\mathbf{q}, s) \geq d_{\min}(n)}} \left(\frac{1}{2} + \varepsilon(n) \right) \Pr[\mathbf{Q} = \mathbf{q}, S = s] \\
&= \left(\frac{1}{2} + \varepsilon(n) \right) \Pr[d(\mathbf{Q}, S) \geq d_{\min}(n)],
\end{aligned}$$

that is,

$$\Pr[\mathcal{D} = B \mid d(\mathbf{Q}, S) \geq d_{\min}(n)] = \frac{1}{2} + \varepsilon(n).$$

Proposition 2. *If $\mathcal{P}$ satisfies the preimage entropy property, then it satisfies the indistinguishability property.*

Proof. Let $\mathbf{Q}$ be the sequence of queries $\mathcal{D}$ makes to PUF, which is a random variable. Notice that

$$\Pr[\mathcal{D} = B] = \Pr[\mathcal{D} = B, d(\mathbf{Q}, S) < d_{\min}(n)] + \Pr[\mathcal{D} = B, d(\mathbf{Q}, S) \geq d_{\min}(n)].$$

From Lemma 1, we have

$$\Pr[d(\mathbf{Q}, S) < d_{\min}(n)] = \varepsilon(n).$$

Furthermore, from Lemma 1, we have

$$\Pr[\mathcal{D} = B, d(\mathbf{Q}, S) \geq d_{\min}(n)] = \left(\frac{1}{2} + \varepsilon(n) \right) \Pr[d(\mathbf{Q}, S) \geq d_{\min}(n)].$$

Thus, we can conclude that

$$\Pr[\mathcal{D} = B] = \varepsilon(n) + \left(\frac{1}{2} + \varepsilon(n) \right) (1 - \varepsilon(n)) = \frac{1}{2} + \varepsilon(n).$$

We now present some analogous properties that are necessary for collective commitment schemes. Let IND_n denote an arbitrary program that generates indices, where $|\text{IND}_n|$ is polynomial in n . The interaction depicted in Fig. 22 is a generalization of the indistinguishability interaction depicted in Fig. 20. Just like before, we say that $\mathcal{P}$ satisfies the collective indistinguishability property if all distinguishers succeed with probability negligibly close to $\frac{1}{2}$.

Lemma 2. *Consider the interaction depicted in Fig. 22. Let $\mathbf{Q}$ be the sequence of queries $\mathcal{D}$ makes to PUF and let $\mathbf{Q}^i := \mathbf{Q} \parallel \mathbf{S}^{j \neq i}$ for each $i \in \text{IND}_n$. Furthermore, we write $\varphi(\mathbf{q}, \mathbf{s})$ instead of*

$$\forall i \in \text{IND}_n \ d(\mathbf{q}^i, \mathbf{s}^i) \geq d_{\min}(n).$$

Then, for all distinguishers $\mathcal{D}$,

$$\Pr[\mathcal{D} = B \mid \varphi(\mathbf{Q}, \mathbf{S})] = \frac{1}{2} + \varepsilon(n).$$

Thus,

$$\begin{aligned}\Pr[\mathcal{D} = B, \varphi(\mathbf{Q}, \mathbf{S})] &= \sum_{\substack{(\mathbf{q}, \mathbf{s}): \\ \varphi(\mathbf{q}, \mathbf{s})}} \Pr[\mathcal{D} = B \mid \mathbf{Q} = \mathbf{q}, \mathbf{S} = \mathbf{s}] \Pr[\mathbf{Q} = \mathbf{q}, \mathbf{S} = \mathbf{s}] \\ &= \sum_{\substack{(\mathbf{q}, \mathbf{s}): \\ \varphi(\mathbf{q}, \mathbf{s})}} \left(\frac{1}{2} + \varepsilon(n) \right) \Pr[\mathbf{Q} = \mathbf{q}, \mathbf{S} = \mathbf{s}] = \left(\frac{1}{2} + \varepsilon(n) \right) \Pr[\varphi(\mathbf{Q}, \mathbf{S})],\end{aligned}$$

that is,

$$\Pr[\mathcal{D} = B \mid \varphi(\mathbf{Q}, \mathbf{S})] = \frac{1}{2} + \varepsilon(n).$$

Lemma 3. *If $\mathcal{P}$ satisfies the preimage entropy property, then it satisfies the collective indistinguishability property.*

Proof. Notice that

$$\Pr[\mathcal{D} = B] = \Pr[\mathcal{D} = B, \neg\varphi(\mathbf{S}, \mathbf{Q})] + \Pr[\mathcal{D} = B, \varphi(\mathbf{S}, \mathbf{Q})],$$

where $\varphi(\mathbf{S}, \mathbf{Q})$ is the event defined in Lemma 2. From Lemma 1, we have

$$\Pr[\neg\varphi(\mathbf{S}, \mathbf{Q})] = \varepsilon(n).$$

Furthermore, from Lemma 2, we have

$$\Pr[\mathcal{D} = B, \varphi(\mathbf{S}, \mathbf{Q})] = \left(\frac{1}{2} + \varepsilon(n) \right) \Pr[\varphi(\mathbf{S}, \mathbf{Q})].$$

Thus, we can conclude that

$$\Pr[\mathcal{D} = B] = \Pr[\mathcal{D} = B, \neg\varphi(\mathbf{S}, \mathbf{Q})] + \Pr[\mathcal{D} = B, \varphi(\mathbf{S}, \mathbf{Q})] = \varepsilon(n) + \left(\frac{1}{2} + \varepsilon(n) \right) (1 - \varepsilon(n)) = \frac{1}{2} + \varepsilon(n).$$

B.3 CRP guessing property

We can formalize the CRP guessing property with an interaction between an honest challenger $\mathcal{C}$ and an adversary $\mathcal{A}$, as depicted in Fig. 24, where $\mathcal{A}$ is successful if $st = \text{Rep}(\text{PUF}(s), p)$ and throughout its execution it does not query PUF on any challenge q such that $\mathbf{d}(q, s) < \mathbf{d}_{\min}(n)$. We say that $\mathcal{P}$ satisfies the CRP guessing property if all adversaries $\mathcal{A}$ in this interaction succeed with negligible probability.

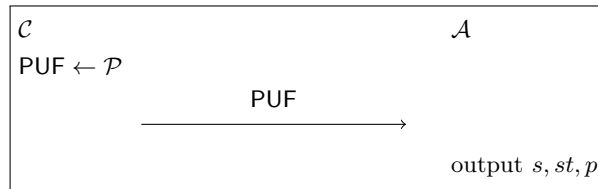


Fig. 24. CRP guessing property interaction.

Although it seems like it can be reduced to the indistinguishability property, we were not able to prove this result. It seems that such a reduction would depend on specific details of the FE, which we want to avoid.

Alternatively, one might consider reducing this requirement to a more fundamental property involving only the PUF family, as depicted in Fig. 25, where $\mathcal{A}$ is successful if σ is a possible response for $\text{PUF}(s)$. However, achieving this reduction is challenging, as it would also rely on the specific characteristics of the FE and PUF family in use. Indeed, even if we could identify some w such that $\text{Gen}(w) = (st, p)$, it would not necessarily enable us to recover an actual possible response σ for $\text{PUF}(s)$.

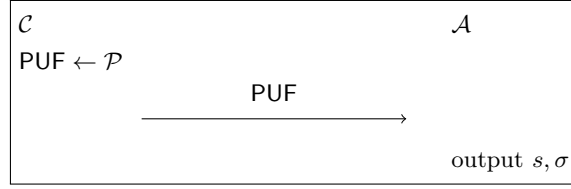


Fig. 25. Alternative CRP guessing property interaction.

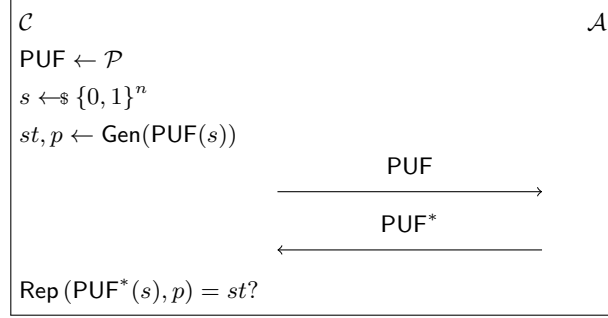


Fig. 26. Test query property interaction.

B.4 TQ property

The TQ property can be formalized with an interaction between an honest challenger $\mathcal{C}$ and an adversary $\mathcal{A}$, as depicted in Fig. 26, where $\mathcal{A}$ is successful if it sends $\text{PUF}^* \neq \text{PUF}$ such that $\text{Rep}(\text{PUF}^*(s), p) = st$. We say that $\mathcal{P}$ satisfies the TQ property if all adversaries $\mathcal{A}$ in this interaction succeed with negligible probability. This is proved in Proposition 3.

Proposition 3. *If $\mathcal{P}$ satisfies the preimage entropy property and the CRP guessing property, then it satisfies the TQ property.*

Proof. Suppose there exists an adversary $\mathcal{A}^0$ that is successful in the TQ interaction with non-negligible probability. In Fig. 27, we construct an adversary $\mathcal{A}$ that contradicts the CRP guessing property.

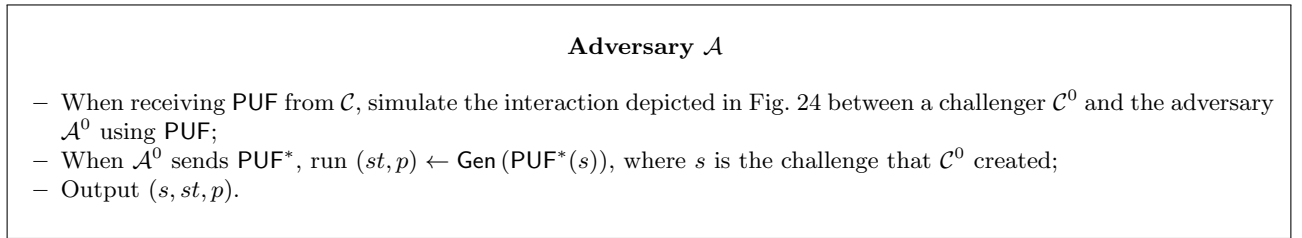


Fig. 27. Reduction to the CRP guessing property.

Let us consider what happens when $\mathcal{A}^0$ is successful and it does not query PUF close to s . Notice that $\mathcal{A}$'s queries are the same as those of $\mathcal{A}^0$ to PUF, along with the additional query $\text{PUF}^*(s)$. Since $\text{PUF}^* \neq \text{PUF}$, we know that $\mathcal{A}$ also does not query PUF close to s . Indeed, this holds because even if PUF^* is malicious, the honest PUFs embedded within it are freshly created using the honest **Sample** algorithm, and therefore distinct from PUF. Moreover, since $\mathcal{A}^0$ is successful, $\mathcal{A}$ outputs a valid CRP. Therefore, under these conditions, $\mathcal{A}$ is always successful.

Now, notice that $\mathcal{A}^0$ is successful with non-negligible probability. Furthermore, from Proposition 1, we know that it only queries PUF close to s with negligible probability. Thus, we can conclude that the probability of both these conditions holding simultaneously is non-negligible, and so $\mathcal{A}$ is successful with non-negligible probability.

C Entropy properties

Consider a random variable X defined on a set D_X . The **max-entropy** and **min-entropy** of X are respectively defined as

$$\begin{aligned} H_0(X) &= \log(|D_X|); \\ H_\infty(X) &= -\log\left(\max_{x \in D_X} \Pr[X = x]\right). \end{aligned}$$

Now, consider another random variable Y , which may be correlated with X . The **average min-entropy** of X given Y is defined as

$$\tilde{H}_\infty(X | Y) = -\log\left(\mathbb{E}_{y \leftarrow Y} \left[\max_{x \in D_X} \Pr[X = x | Y = y] \right]\right).$$

Lemma 4. *For any function f and random variables X, Y , we have $\tilde{H}_\infty(X | Y) \leq \tilde{H}_\infty(X | f(Y))$.*

Proof. We want to show that

$$-\log \mathbb{E}_{y \leftarrow Y} \left[\max_{x \in D_X} \Pr[X = x | Y = y] \right] \leq -\log \mathbb{E}_{z \leftarrow f(Y)} \left[\max_{x \in D_X} \Pr[X = x | f(Y) = z] \right],$$

which is equivalent to

$$\mathbb{E}_{y \leftarrow Y} \left[\max_{x \in D_X} \Pr[X = x | Y = y] \right] \geq \mathbb{E}_{z \leftarrow f(Y)} \left[\max_{x \in D_X} \Pr[X = x | f(Y) = z] \right].$$

For each $z \in D_{f(Y)}$, let x_z^* be such that

$$\Pr[X = x_z^* | f(Y) = z] = \max_{x \in D_X} \Pr[X = x | f(Y) = z].$$

Thus,

$$\begin{aligned} \mathbb{E}_{y \leftarrow Y} \left[\max_{x \in D_X} \Pr[X = x | Y = y] \right] &\geq \mathbb{E}_{y \leftarrow Y} \left[\Pr[X = x_{f(y)}^* | Y = y] \right] \\ &= \sum_{y \in D_Y} \Pr[Y = y] \Pr[X = x_{f(y)}^* | Y = y] = \sum_{y \in D_Y} \Pr[X = x_{f(y)}^*, Y = y] \\ &= \sum_{z \in D_{f(Y)}} \sum_{y \in f^{-1}(z)} \Pr[X = x_z^*, Y = y] = \sum_{z \in D_{f(Y)}} \Pr[X = x_z^*, f(Y) = z] \\ &= \sum_{z \in D_{f(Y)}} \Pr[f(Y) = z] \Pr[X = x_z^* | f(Y) = z] \\ &= \mathbb{E}_{z \leftarrow f(Y)} \left[\max_{x \in D_X} \Pr[X = x | f(Y) = z] \right]. \end{aligned}$$

Lemma 5. *The following properties of the average min-entropy regarding independence hold for any random variables X, Y, Z :*

- If $(X, Y) \perp\!\!\!\perp Z$, then $\tilde{H}_\infty(X | Y, Z) = \tilde{H}_\infty(X | Y)$;
- If $X \perp\!\!\!\perp Z$, then $\tilde{H}_\infty(X | Z) = H_\infty(X)$.

Proof. To prove the first property, notice that $(X, Y) \perp\!\!\!\perp Z$ implies that for all x, y, z ,

$$\Pr[X = x | Y = y, Z = z] = \Pr[X = x | Y = y].$$

Thus,

$$\begin{aligned} \tilde{H}_\infty(X | Y, Z) &= -\log\left(\mathbb{E}_{(y,z) \leftarrow (Y,Z)} \left[\max_{x \in D_X} \Pr[X = x | Y = y, Z = z] \right]\right) \\ &= -\log\left(\mathbb{E}_{(y,z) \leftarrow (Y,Z)} \left[\max_{x \in D_X} \Pr[X = x | Y = y] \right]\right) \\ &= -\log\left(\mathbb{E}_{y \leftarrow Y} \left[\max_{x \in D_X} \Pr[X = x | Y = y] \right]\right) \\ &= \tilde{H}_\infty(X | Y). \end{aligned}$$

The second property is a particular case of the first one, where Y is constant.

Lemma 6. Let A be a random variable and X_A be a random variable that is parametrized on A . Furthermore, let Y be another random variable and suppose $\tilde{H}_\infty(X_a | Y) = H$ for all $a \in D_A$. Then,

$$\tilde{H}_\infty(X_A | Y) \geq H - H_0(A).$$

Proof. We just have to notice that

$$\begin{aligned} H_\infty(X_A) &= -\log \left(\mathbb{E}_{y \leftarrow Y} \left[\max_{x \in D_{X_A}} \Pr[X_A = x | Y = y] \right] \right) \\ &= -\log \left(\mathbb{E}_{y \leftarrow Y} \left[\max_{x \in D_{X_A}} \sum_{a \in D_A} \Pr[X_a = x, A = a | Y = y] \right] \right) \geq -\log \left(\mathbb{E}_{y \leftarrow Y} \left[\max_{x \in D_{X_A}} \sum_{a \in D_A} \Pr[X_a = x | Y = y] \right] \right) \\ &\geq -\log \left(\mathbb{E}_{y \leftarrow Y} \left[\sum_{a \in D_A} \max_{x \in D_{X_A}} \Pr[X_a = x | Y = y] \right] \right) = -\log \left(\sum_{a \in D_A} \mathbb{E}_{y \leftarrow Y} \left[\max_{x \in D_{X_A}} \Pr[X_a = x | Y = y] \right] \right) \\ &= -\log \left(\sum_{a \in D_A} 2^{-\tilde{H}_\infty(X_a | Y)} \right) = -\log(|D_A| 2^{-H}) = -\log(2^{-H}) - \log(|D_A|) = H - H_0(A). \end{aligned}$$

Lemma 7. The following weak chain rule holds for any random variables X, Y :

$$\tilde{H}_\infty(X | Y, Z) \geq \tilde{H}_\infty(X | Y) - H_0(Z).$$

Furthermore, as a consequence,

$$\tilde{H}_\infty(X | Z) \geq H_\infty(X) - H_0(Z).$$

Proof. It follows from Lemma 2.2 of [20].

Lemma 8. Let X and Y be random variables defined on the same set D . Then,

$$\Pr[X = Y] \leq 2^{-\tilde{H}_\infty(X | Y)}.$$

Proof. As noted in [21],

$$\begin{aligned} \Pr[X = Y] &= \sum_{y \in D} \Pr[X = y, Y = y] = \sum_{y \in D} \Pr[X = y | Y = y] \Pr[Y = y] \\ &= \mathbb{E}_{y \leftarrow Y} [\Pr[X = y | Y = y]] \leq \mathbb{E}_{y \leftarrow Y} \left[\max_{x \in D} \Pr[X = x | Y = y] \right] \\ &= 2^{-\tilde{H}_\infty(X | Y)}. \end{aligned}$$

Lemma 9. Let X and Y be random variables defined on the same set D . Furthermore, consider a family of sets $A(d) \subseteq D$ for each $d \in D$ such that:

- $|A(d)|$ does not depend on d (and so we write $|A|$ instead);
- $\bigcup_{d \in D} A(d) = D$.

Then,

$$\Pr[X \in A(Y)] \leq |A| 2^{-\tilde{H}_\infty(X | Y)}.$$

Remark 2. Notice that Lemma 8 is a particular case of this one, where $A(d) = \{d\}$, for $d \in D$.

Proof.

$$\begin{aligned} \Pr[X \in A(Y)] &= \sum_{y \in D} \Pr[X \in A(y) | Y = y] \Pr[Y = y] = \mathbb{E}_{y \leftarrow Y} [\Pr[X \in A(y) | Y = y]] \\ &\leq \mathbb{E}_{y \leftarrow Y} \left[\max_{x \in D} \Pr[X \in A(x) | Y = y] \right] = \mathbb{E}_{y \leftarrow Y} \left[\max_{x \in D} \sum_{x' \in A(x)} \Pr[X = x' | Y = y] \right] \\ &\leq \mathbb{E}_{y \leftarrow Y} \left[\max_{x \in D} \sum_{x' \in A(x)} \max_{x' \in A(x)} \Pr[X = x' | Y = y] \right] = \mathbb{E}_{y \leftarrow Y} \left[\max_{x \in D} |A| \max_{x' \in A(x)} \Pr[X = x' | Y = y] \right] \\ &= |A| \mathbb{E}_{y \leftarrow Y} \left[\max_{x \in D} \max_{x' \in A(x)} \Pr[X = x' | Y = y] \right] = |A| \mathbb{E}_{y \leftarrow Y} \left[\max_{x' \in \bigcup_{x \in D} A(x)} \Pr[X = x' | Y = y] \right] \\ &= |A| \mathbb{E}_{y \leftarrow Y} \left[\max_{x' \in D} \Pr[X = x' | Y = y] \right] = |A| 2^{-\tilde{H}_\infty(X | Y)}. \end{aligned}$$

D A possible fix to ExtPUF

One possible approach to avoiding the attack described in Section 4.1 is to modify the original ExtPUF protocol so that S sends c_2 *before* receiving r_1 . In other words, S completes the commitment to $st_E \parallel p_E$ before learning r_1 . This modification is illustrated in Fig. 28. In this case, the extractor would operate as before, except that it would only consider the queries made to PUF_E *prior to the transmission of r_1* . Consequently, S^* would only be able to query PUF_E on $Enc(st_1 \oplus r_1)$ when it's already too late for the extractor to see it, and thus the attack would fail.

Unfortunately, despite its intuitive appeal, there are some significant challenges when it comes to proving extractability, particularly in the case where the extractor is unable to extract any string. To see the issue, recall that for S^* to decommit successfully, it must hold that:

$$st_E = \text{Rep}(PUF_E(\text{Enc}(st_1)), p_E).$$

This seems to imply that S^* correctly computed $PUF_E(\text{Enc}(st_1))$ without issuing any queries close to $\text{Enc}(st_1)$, which according to the CRP guessing property should only happen with negligible probability. However, the extractor's view is limited to pre- r_1 queries: it remains possible that S^* queried PUF_E on $\text{Enc}(st_1)$ after receiving r_1 , and thus after sending c_2 . Since c_2 corresponds to the commitment $\text{CPUF}(st_E \parallel p_E)$, this would imply that S^* committed to a value it could only guess with negligible probability and yet it later decommits successfully.

We can abstract this into the following two-phase scenario: in the first phase, the adversary commits (via CUPF) to a string x that it does not know – having only negligible probability of guessing it. In the second phase, the adversary learns x and attempts to decommit to it. Despite not knowing x at the time of commitment, it is somehow able to open the commitment successfully. Intuitively, this seems to violate the binding property of CUPF, but we have not been able to formally prove this. In fact, it's unclear whether such a guarantee should follow solely from binding. Proving it may require stronger properties, such as extractability – which, of course, CUPF does not provide.

It is also worth noting that this issue is not introduced by this modified protocol. The original version from [12] is also affected, though in that case, it is overshadowed by even more severe limitations. Furthermore, aside from this unresolved issue, we were able to prove the security of the modified ExtPUF protocol.

E Security proofs of the commitment schemes

Theorem 1. *CUPF is an ideal commitment scheme in the $\mathcal{F}_{\text{MPUF}}$ -hybrid model with unbounded k_{state} .*

Proof. Completeness clearly follows from the response consistency property 3.1, and thus its proof is omitted. We will do the same for the remaining proofs.

Computationally Hiding: Suppose, by contradiction, that this is not the case. Then, there exist different strings x^0 and x^1 and a malicious receiver R^* such that, in the interaction depicted in Fig. 13,

$$\Pr\left[R^*\left(\text{Commit}^{\text{CUPF}}\left(x^{B^0}\right)\right) = B^0\right] - \frac{1}{2}$$

is not negligible²³.

Consider a modified protocol CUnif²⁴, where S uses $u \leftarrow \{0, 1\}^{kl}$ instead of st . That is, in that protocol, S does $c := u \oplus (x^l \wedge r)$. Then,

$$\Pr\left[R^*\left(\text{Commit}^{\text{CUnif}}\left(x^{B^0}\right)\right) = B^0\right] = \frac{1}{2},$$

because the distribution of the interaction is independent from B^0 .

Now, consider the interaction depicted in Fig. 20 between a challenger $\mathcal{C}$ and a distinguisher $\mathcal{D}$, corresponding to the indistinguishability property of PUFs. In Fig. 29, we present a distinguisher $\mathcal{D}$ that breaks this property, thereby contradicting Lemma 2. Here, CBS denotes a protocol just like CUPF, but where S does not create its PUF and does not need to query PUF to get st and p . Instead, S receives bs, p, PUF from $\mathcal{D}$ (who receives them from $\mathcal{C}$) and uses bs instead of st in c .

²³ We denote the random variable as B^0 in this interaction to avoid confusion with the analogous B in the subsequent interaction we define.

²⁴ Given that u is chosen uniformly, the decommitment phase of CUnif is actually not defined. This is not a problem, because we will only run its commitment phase.

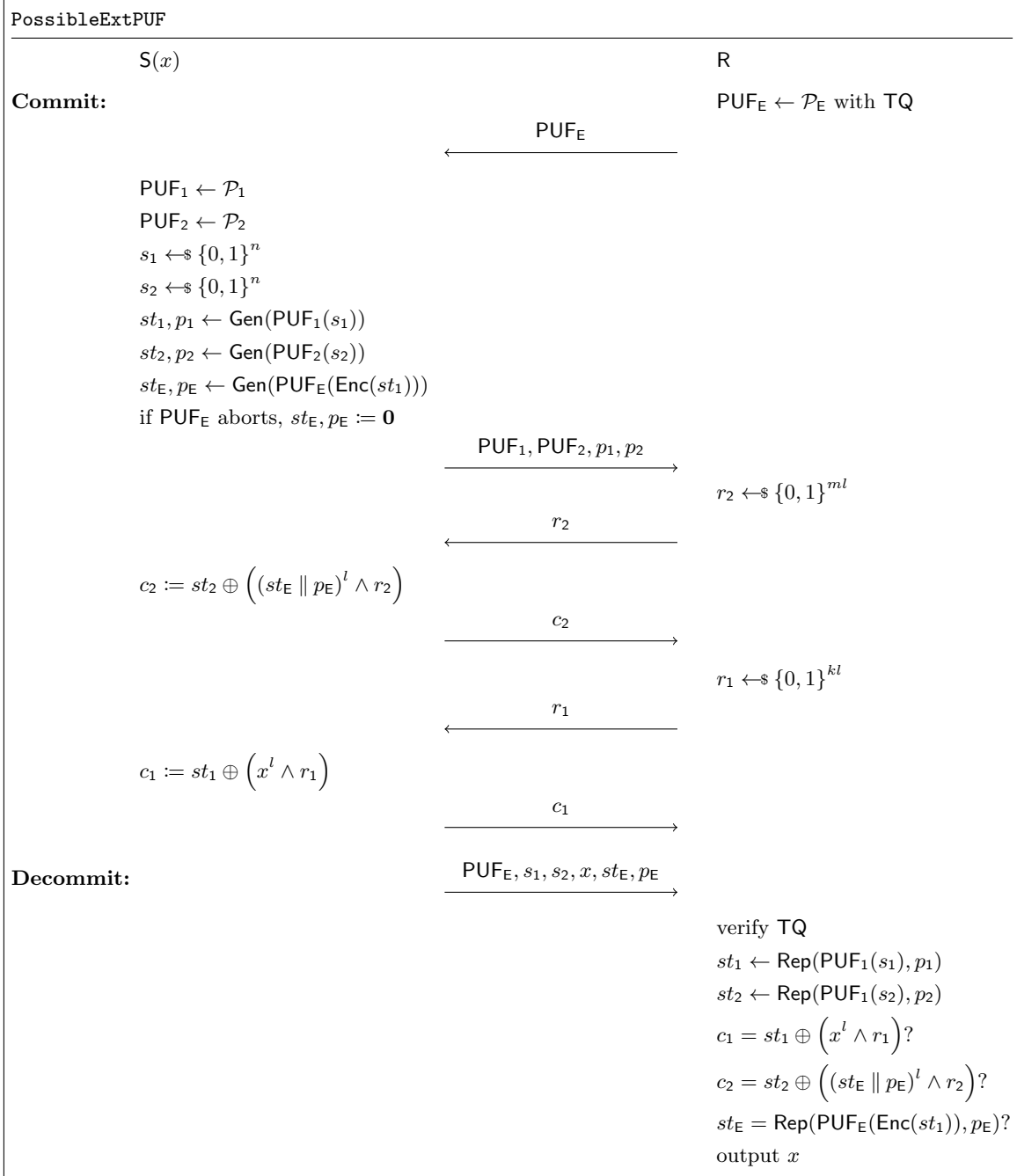


Fig. 28. A possible solution to ExtPUF.

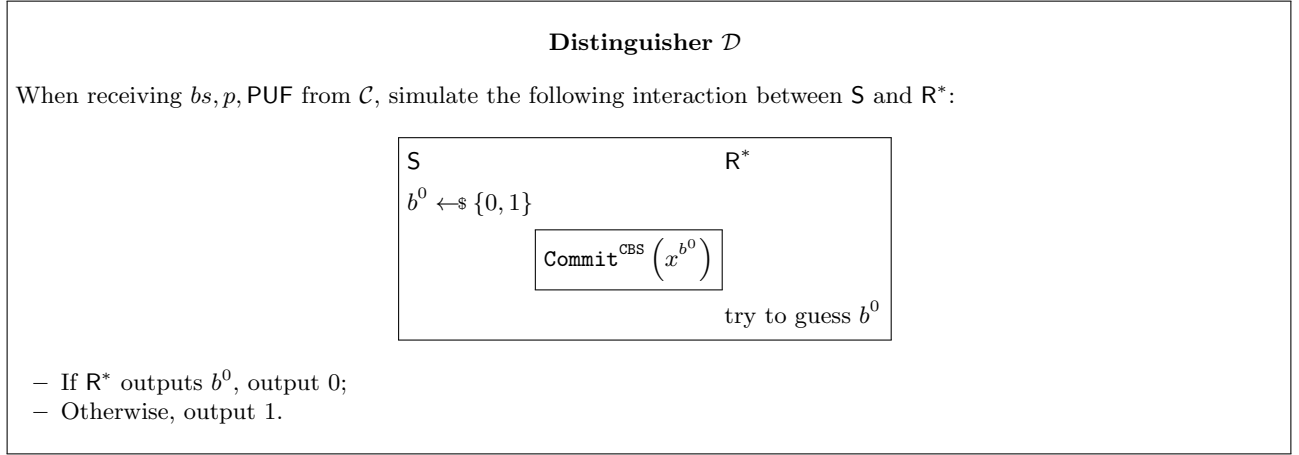


Fig. 29. Distinguisher that breaks the indistinguishability property.

Notice that if $b = 0$, then $bs = st$, making **CBS** identical to **CPUF**. Likewise, if $b = 1$, then $bs = u$, and so **CBS** is identical to **CUnif**. Thus,

$$\begin{aligned}
2 \Pr[\mathcal{D} = B] &= \Pr[\mathcal{D} = 0 \mid B = 0] + \Pr[\mathcal{D} = 1 \mid B = 1] \\
&= \Pr\left[\mathbf{R}^* \left(\text{Commit}^{\text{CPUF}}(x^{B^0}) \right) = B^0\right] + \Pr\left[\mathbf{R}^* \left(\text{Commit}^{\text{CUnif}}(x^{B^0}) \right) \neq B^0\right] \\
&= \Pr\left[\mathbf{R}^* \left(\text{Commit}^{\text{CPUF}}(x^{B^0}) \right) = B^0\right] + 1 - \frac{1}{2} = \Pr\left[\mathbf{R}^* \left(\text{Commit}^{\text{CPUF}}(x^{B^0}) \right) = B^0\right] + \frac{1}{2},
\end{aligned}$$

and so

$$\Pr[\mathcal{D} = B] - \frac{1}{2} = \frac{1}{2} \left(\Pr\left[\mathbf{R}^* \left(\text{Commit}^{\text{CPUF}}(x^{B^0}) \right) = B^0\right] + \frac{1}{2} \right) - \frac{1}{2} = \frac{1}{2} \left(\Pr\left[\mathbf{R}^* \left(\text{Commit}^{\text{CPUF}}(x^{B^0}) \right) = B^0\right] - \frac{1}{2} \right)$$

is non-negligible, which is a contradiction.

Statistically Binding: Suppose $\mathbf{S}^*$ has some malicious behavior and makes a certain commitment. We have to show that the probability of $\mathbf{S}^*$ being able to open that commitment to different strings X and Y successfully is negligible. When decommitting to X , $\mathbf{S}^*$ sends S , and $\mathbf{R}$ runs $\text{Rep}(\text{PUF}(S), P)$, with this final expression being a random variable that depends on²⁵:

1. the random variable S ;
2. PUF 's state **STATE** in that moment and its internal randomness, which is independent from R ;
3. the random variable P , which is independent from R ;
4. the randomness of the fuzzy extractor, which is independent from R .

Therefore, we can write $\text{Rep}(\text{PUF}(S), P) = f(S, \text{STATE}, T)$, where T represents some internal randomness that is independent from the remaining variables. Thus, if the decommitment is successful,

$$C = f(S, \text{STATE}, T) \oplus (X^l \wedge R).$$

Likewise, for the decommitment of Y , we can write $\text{Rep}(\text{PUF}(S'), P) = g(S', \text{STATE}', T')$, where T' is also independent from the remaining variables. If the decommitment is successful,

$$C = g(S', \text{STATE}', T') \oplus (Y^l \wedge R).$$

Suppose X and Y differ on an index J . Given $j \in [k]$, let $\mathbf{I}_j$ be the set of the positions of the commitment C that are used for committing the j -th bit of the string. That is, $\mathbf{I}_j = \{j, j+k, \dots, j+(l-1)k\}$. Then,

$$C_{\mathbf{I}_j} = f(S, \text{STATE}, T)_{\mathbf{I}_j} \oplus (X_J^l \wedge R_{\mathbf{I}_j}) = g(S', \text{STATE}', T')_{\mathbf{I}_j} \oplus (Y_J^l \wedge R_{\mathbf{I}_j}),$$

²⁵ The use of capital letters is deliberate. Here, instead of writing the values as x, s, p like in the protocol, we write X, S, P to emphasize the fact that they are random variables. We adopt this convention throughout our work.

which implies

$$R_{I_J} = f(S, \text{STATE}, T)_{I_J} \oplus g(S', \text{STATE}', T')_{I_J},$$

since $X_J \neq Y_J$. Hence, if we show that

$$\Pr[R_{I_J} = f(S, \text{STATE}, T)_{I_J} \oplus g(S', \text{STATE}', T')_{I_J}]$$

is negligible, we are done. Let STATE_0 be PUF's state when it is sent to R. Notice that $\text{STATE} = \text{STATE}' = \text{STATE}_0$, given that PUF is not queried up to the moment of decommitment. Furthermore, $\text{STATE}_0 \perp\!\!\!\perp R$, since R is only sent after PUF. Therefore,

$$\begin{aligned} & \tilde{H}_\infty(R_{I_J} \mid f(S, \text{STATE}, T)_{I_J} \oplus g(S', \text{STATE}', T')_{I_J}) \\ &= \tilde{H}_\infty(R_{I_J} \mid f(S, \text{STATE}_0, T)_{I_J} \oplus g(S', \text{STATE}_0, T')_{I_J}) \\ &\geq \tilde{H}_\infty(R_{I_J} \mid \text{STATE}_0, J, S, S', T, T') \quad (\text{Lemma 4}) \\ &= \tilde{H}_\infty(R_{I_J} \mid \text{STATE}_0, J, S, S') \quad ((T, T') \perp\!\!\!\perp (R_{I_J}, J, \text{STATE}_0, S, S') \text{ and Lemma 5}) \\ &\geq \tilde{H}_\infty(R_{I_J} \mid \text{STATE}_0) - H_0(J, S, S', J). \quad (\text{Lemma 7}) \end{aligned}$$

Now, although $\text{STATE}_0 \perp\!\!\!\perp R$, this does not necessarily imply $\text{STATE}_0 \perp\!\!\!\perp R_{I_J}$. Indeed, R_{I_J} depends on J , which in turn can depend on STATE_0 . However, we still know that $\text{STATE}_0 \perp\!\!\!\perp R_{I_j}$ for each $j \in [k]$. Therefore, by Lemma 5, for each $j \in [k]$,

$$\tilde{H}_\infty(R_{I_j} \mid \text{STATE}_0) = H_\infty(R_{I_j}) = l(n),$$

which implies

$$\begin{aligned} \tilde{H}_\infty(R_{I_J} \mid \text{STATE}_0) - H_0(J, S, S', J) &\geq l(n) - H_0(J) - H_0(J, S, S', J) \quad (\text{Lemma 6}) \\ &= 3n - 2\log(k) - 2n \\ &= n - 2\log(k). \end{aligned}$$

Thus, by Lemma 8,

$$\Pr[R_I = f(S, \text{STATE}, T)_I \oplus g(S', \text{STATE}', T')_I] \leq 2^{-n+2\log(k)},$$

which is negligible.

Theorem 2. *NotifExtPUF is an ideal extractable commitment scheme in the $\mathcal{F}_{\text{MPUFNotif-hybrid}}$ model.*

Proof. Let $I_j := \{j, j+k, \dots, j+(n-1)k\}$, which is the set of positions of c that are used for committing the j -th bit of x .

Computationally Hiding: A similar argument to the one used for CUPF can be used here. In the similarly defined NotifExtUnif protocol, we know the information R^* receives is independent from B^0 because S does not abort when PUF_E aborts.

Statistically Binding: Suppose S^* decommits successfully with PUF_E, S, X, ST_E, P_E . From Lemma 3, we know that S^* returned the same PUF_E with overwhelming probability, so we can assume that is the case.

First, we will show that with overwhelming probability S^* queried PUF_E on some Q_X that is close to $Q = \text{Enc}(ST)^{26}$, where ST is the one obtained by R in the decommitment. Indeed, suppose that, with non-negligible probability, none of the queries were close to Q . In that case, we can define an adversary $\mathcal{A}$, depicted in Fig. 30, which contradicts the CRP guessing property depicted in Fig. 24, when interacting with a challenger $\mathcal{C}$. We know that

- with non-negligible probability, S^* does not query PUF_E with queries that are close to Q , which implies that the same happens for $\mathcal{A}$;
- S^* decommits successfully to X , which implies that $\mathcal{A}$ can output Q, ST, P such that $ST = \text{Rep}(\text{PUF}(Q), P)$,

and so $\mathcal{A}$ is successful with non-negligible probability.

²⁶ That is, $d(Q_X, Q) < d_{\min}$.

Adversary $\mathcal{A}$

- When receiving PUF_E from $\mathcal{C}$, simulate **ExtPUF** between the malicious S^* and the honest R , who uses PUF_E ;
- When S^* decommits with s, x, st_E, p_E , output $\text{Enc}(st), st_E, p_E$, where st is the one obtained by R in the decommitment.

Fig. 30. Adversary that breaks the CRP guessing property.

Therefore, we have shown that if S^* decommits successfully with $\text{PUF}_E, S, X, ST_E, P_E$, then with overwhelming probability it queries PUF_E on some Q_X such that

$$C = ST \oplus (X^n \wedge R) = \text{Dec}(Q_X) \oplus (X^n \wedge R).$$

Likewise, if S^* can also decommit successfully with $\text{PUF}_E, S', Y, ST'_E, P'_E$, then with overwhelming probability it queries PUF_E on some Q_Y such that

$$C = ST' \oplus (Y^n \wedge R) = \text{Dec}(Q_Y) \oplus (Y^n \wedge R).$$

where ST' is the one obtained by R in the decommitment. Suppose X and Y differ on an index J (which is also a random variable). Then,

$$C_{I_J} = \text{Dec}(Q_X)_{I_J} \oplus (X_J^n \wedge R_{I_J}) = \text{Dec}(Q_Y)_{I_J} \oplus (Y_J^n \wedge R_{I_J}),$$

and so

$$R_{I_J} = \text{Dec}(Q_X)_{I_J} \oplus \text{Dec}(Q_Y)_{I_J}.$$

Recall that R aborts if S^* queries PUF_E after sending R . Since the decommitment is successful, this must mean that the queries Q_X and Q_Y were done by S^* before receiving R , which implies $(Q_X, Q_Y) \perp\!\!\!\perp R$ – this is precisely where the changes we made to the PUF model and the protocol come into play. Just like in **CPUF**, this does not necessarily imply $(Q_X, Q_Y) \perp\!\!\!\perp R_I$, but we still have $(Q_X, Q_Y) \perp\!\!\!\perp R_{I_j}$ for each $j \in [k]$. Therefore, by Lemma 5, for each $j \in [k]$,

$$\tilde{H}_\infty(R_{I_j} \mid Q_X, Q_Y) = H_\infty(R_{I_j}) = n,$$

which implies

$$\begin{aligned} \tilde{H}_\infty(R_{I_J} \mid \text{Dec}(Q_X)_{I_J} \oplus \text{Dec}(Q_Y)_{I_J}) &\geq \tilde{H}_\infty(R_{I_J} \mid Q_X, Q_Y, J) && \text{(Lemma 4)} \\ &\geq \tilde{H}_\infty(R_{I_J} \mid Q_X, Q_Y) - H_0(J) && \text{(Lemma 7)} \\ &\geq n - H_0(J) - H_0(J) && \text{(Lemma 6)} \\ &= n - 2 \log(k). \end{aligned}$$

Thus, by Lemma 8,

$$\Pr[R_{I_J} = \text{Dec}(Q_X)_{I_J} \oplus \text{Dec}(Q_Y)_{I_J}] \leq 2^{-n+2 \log(k)},$$

which is negligible.

Extractability: Consider the extractor depicted in Figure 31, which is an adapted version of the original extractor. Notice that x is extracted from a query q if and only if $r_{I_j} \neq \mathbf{0}$ for all $j \in [k]$ and $c = \text{Dec}(q) \oplus (x^n \wedge r)$.

E is clearly PPT and verifies the simulation property. Now, we want to prove the extraction property, that is,

$$\Pr[S^* \text{ decommits successfully to } X \neq X^*] = \varepsilon(n).$$

Suppose S^* successfully decommits with $\text{PUF}_E, S, X, ST_E, P_E$. As argued in binding, we know that with overwhelming probability S^* queried PUF_E on some Q_X such that $C = \text{Dec}(Q_X) \oplus (X^n \wedge R)$ before receiving R . Furthermore, notice that with overwhelming probability $R_{I_j} \neq \mathbf{0}$ for all $j \in [k]$. Therefore, we know that with overwhelming probability E extracted X from Q_X .

Additionally, we also know that S^* made no additional queries after receiving R , since the decommitment is successful. Consequently, the only way that E could output $X^* \neq X$ is if it also extracted some string $Y \neq X$ from some query Q_Y , in which case $X^* = \perp$. However, that would imply

$$C = \text{Dec}(Q_X) \oplus (X^n \wedge R) = \text{Dec}(Q_Y) \oplus (Y^n \wedge R),$$

Extractor E

E proceeds like an honest receiver R, while also saving S^* 's queries to PUF_E in $\mathcal{Q}$. Once the commitment phase is completed, it does the following:

- If S^* queried PUF_E after receiving r , output $x^* = \perp$;
- Otherwise, for each $q \in \mathcal{Q}$, try to extract a string x from q by running $\text{ExtractFromQuery}(\text{Dec}(q))$, where:

```

ExtractFromQuery( $st$ )
 $x := \varepsilon$ 
for  $j \in [k]$  :
  if  $c_{I_j} = st_{I_j} \wedge c_{I_j} \neq st_{I_j} \oplus r_{I_j}$  :
     $x := x \parallel 0$ 
  elseif  $c_{I_j} = st_{I_j} \oplus r_{I_j} \wedge c_{I_j} \neq st_{I_j}$  :
     $x := x \parallel 1$ 
  else :
    return  $\perp$ 
return  $x$ 

```

- If exactly one string x was extracted, output $x^* = x$;
- Otherwise, output $x^* = \perp$.

Fig. 31. The extractor for NotifExtPUF .

which only happens with negligible probability, as argued in binding. Thus, we conclude that

$$\Pr[S^* \text{ decommits successfully to } X \neq X^*] = \varepsilon(n).$$

Theorem 3. *StatelessExtPUF is an ideal extractable commitment scheme in the $\mathcal{F}_{\text{MPUF-hybrid}}$ model with $k_{\text{state}} = 0$.*

Proof. Computationally Hiding: A similar argument to the one used for CUPF can be used here. In the similarly defined StatelessExtUnif protocol, we know the information R^* receives is independent from B^0 because S does not abort when PUF_E aborts and because PUF_E is stateless.

Statistically Binding: Analogous to Theorem 2.

Extractability: Analogous to Theorem 2. The extractor is depicted in Fig. 32.

The collective version of NotifExtPUF , CollNotifExtPUF , is depicted in Fig. 33.

Theorem 5. *CollNotifExtPUF is an ideal extractable collective commitment in the $\mathcal{F}_{\text{MPUFNotif-hybrid}}$ model.*

Proof. Computationally Hiding: Suppose, by contradiction, that this is not the case. Then, there exists an interaction INTER and a malicious receiver R^* such that, for the interaction depicted in Fig. 7,

$$\Pr[R^*(\text{INTER}^{\text{CollNotifExtPUF}}(W)) = W] - \frac{1}{|\Omega_n|}$$

is not negligible. Consider a modified protocol CollNotifExtUnif ,²⁷ where S uses $u^i \leftarrow \$ \{0,1\}^{kn}$ instead of st^i , for each $i \in \text{CLOSED}_n$. Then,

$$\Pr[R^*(\text{INTER}^{\text{CollNotifExtUnif}}(W)) = W] = \frac{1}{|\Omega_n|},$$

because the distribution of the interaction is independent from W . Indeed, notice that, although S^* returned all the PUF_E^i for each $i \in \text{OPEN}_n$, these PUFs were not queried on information that depends on W . Furthermore, S does not abort when any PUF_E^i aborts. Therefore, R^* does not get any information that depends on W .

²⁷ Again, the decommitments of CollNotifExtUnif are actually not defined for $i \in \text{CLOSED}_n$. This is not a problem, because we will only run its commitment phase and decommitments for $i \in \text{OPEN}_n$.

Extractor E

E proceeds like an honest R, while also saving S^* 's queries to PUF_E in $\mathcal{Q}$. Once the commitment phase is completed, for each $q \in \mathcal{Q}$, it tries to extract a string x from q by running $\text{ExtractFromQuery}(\text{Dec}(q))$, where:

```

ExtractFromQuery( $st$ )
 $x := \varepsilon$ 
for  $j \in [k]$  :
    if  $c_{I_j} = st_{I_j} \wedge c_{I_j} \neq st_{I_j} \oplus r_{I_j}$  :
         $x := x \parallel 0$ 
    elseif  $c_{I_j} = st_{I_j} \oplus r_{I_j} \wedge c_{I_j} \neq st_{I_j}$  :
         $x := x \parallel 1$ 
    else :
        return  $\perp$ 
return  $x$ 

```

Then, it does the following:

- If exactly one string x was extracted, output $x^* = x$;
- Otherwise, output $x^* = \perp$.

Fig. 32. The extractor for **StatelessExtPUF**.

Now, consider the interaction depicted in Fig. 22 between a challenger $\mathcal{C}$ and a distinguisher $\mathcal{D}$, corresponding to a generalized indistinguishability property of PUFs. In Fig. 34, we present a distinguisher $\mathcal{D}$ that breaks this property, thereby contradicting Lemma 3, with $\text{IND}_n = \text{CLOSED}_n$. Here, **CollNotifExtBS** denotes a protocol just like **CollNotifExtPUF**, but where:

- S receives PUF and bs^i, p^i for each $i \in \text{CLOSED}_n$ from $\mathcal{D}$ (who received them from $\mathcal{C}$) and uses each bs^i instead of st^i ;
- S generates the st^i, p^i for each $i \in \text{OPEN}_n$ using PUF.

Notice that if $b = 0$, then $bs^i = st^i$ for each $i \in \text{CLOSED}_n$, making **CollNotifExtBS** identical to **CollNotifExtPUF**. Likewise, if $b = 1$, then $bs^i = u^i$ for each $i \in \text{CLOSED}_n$, and so **CollNotifExtBS** is identical to **CollNotifExtUnif**. Thus,

$$\begin{aligned}
 2 \Pr[\mathcal{D} = B] &= \Pr[\mathcal{D} = 0 \mid B = 0] + \Pr[\mathcal{D} = 1 \mid B = 1] \\
 &= \Pr[\mathbf{R}^* (\text{INTER}^{\text{CollNotifExtPUF}}(W)) = W] + \Pr[\mathbf{R}^* (\text{INTER}^{\text{CollNotifExtUnif}}(W)) \neq W] \\
 &= \Pr[\mathbf{R}^* (\text{INTER}^{\text{CollNotifExtPUF}}(W)) = W] + 1 - \frac{1}{|\Omega_n|},
 \end{aligned}$$

and so

$$\Pr[\mathcal{D} = B] - \frac{1}{2} = \frac{1}{2} \left(\Pr[\mathbf{R}^* (\text{INTER}^{\text{CollNotifExtPUF}}(W)) = W] - \frac{1}{|\Omega_n|} \right)$$

is non-negligible, which is a contradiction.

Statistically Binding: Suppose a malicious sender S^* and an honest receiver interact according to some **INTER**, which involves S^* making a commitment. Furthermore, suppose S^* is able to open some commitment I successfully to strings $X^I \neq Y^I$. Just like in Theorem 2, we can show that if S^* decommits I successfully to X^I , then with overwhelming probability S^* queried PUF_E^I on some Q_{X^I} that is close to $Q = \text{Enc}(ST^I)$ before receiving R^I , where ST^I is the one R obtained in the decommitment of I . Likewise, if it decommits I successfully to Y^I , then with overwhelming probability S^* queried PUF_E^I on some Q_{Y^I} that is close to $Q' = \text{Enc}((ST')^I)$ before receiving R^I , where $(ST')^I$ is the one R obtained in the decommitment of I . Suppose X^I and Y^I differ on an index J . Again,

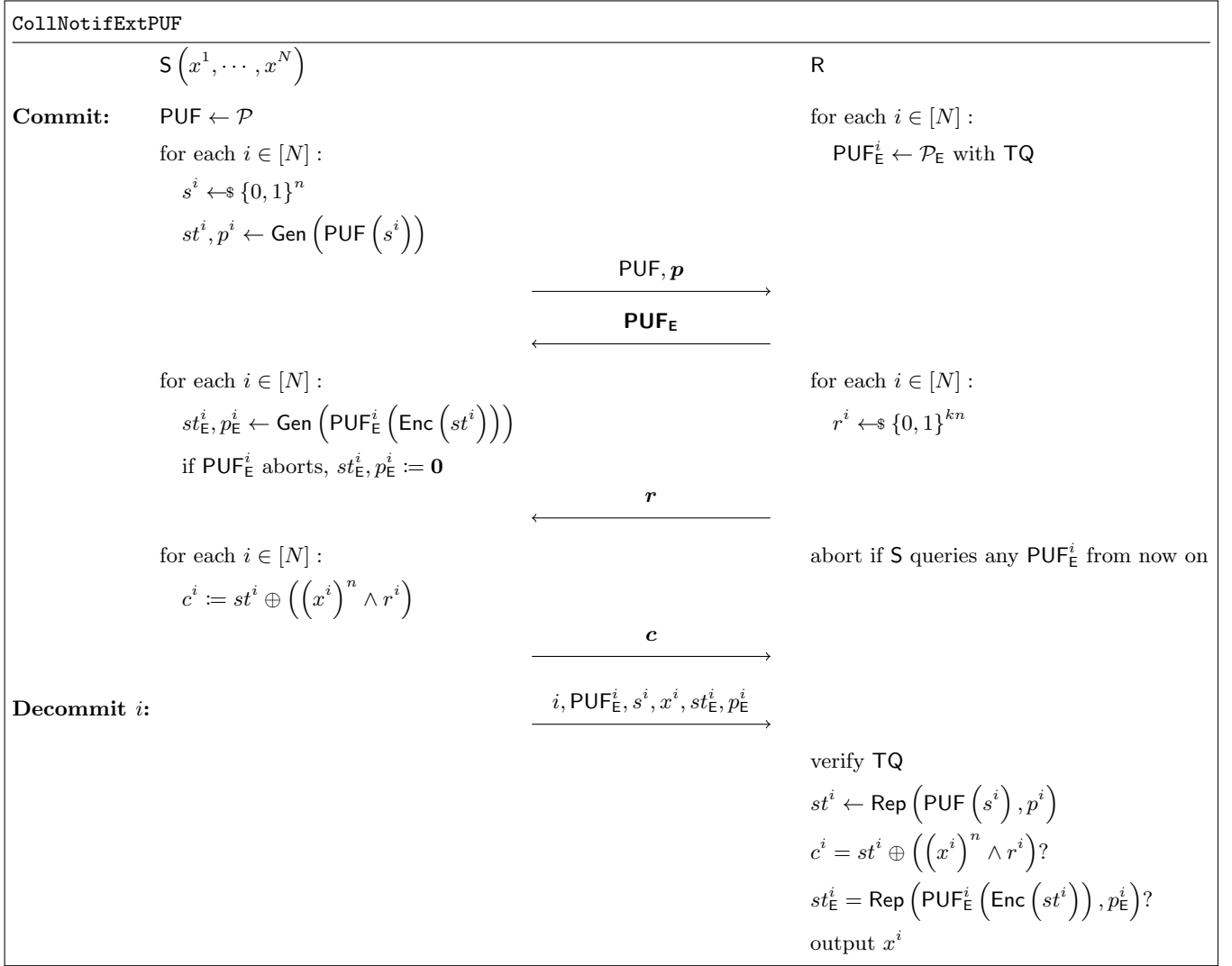


Fig. 33. The CollNotifExtPUF protocol.

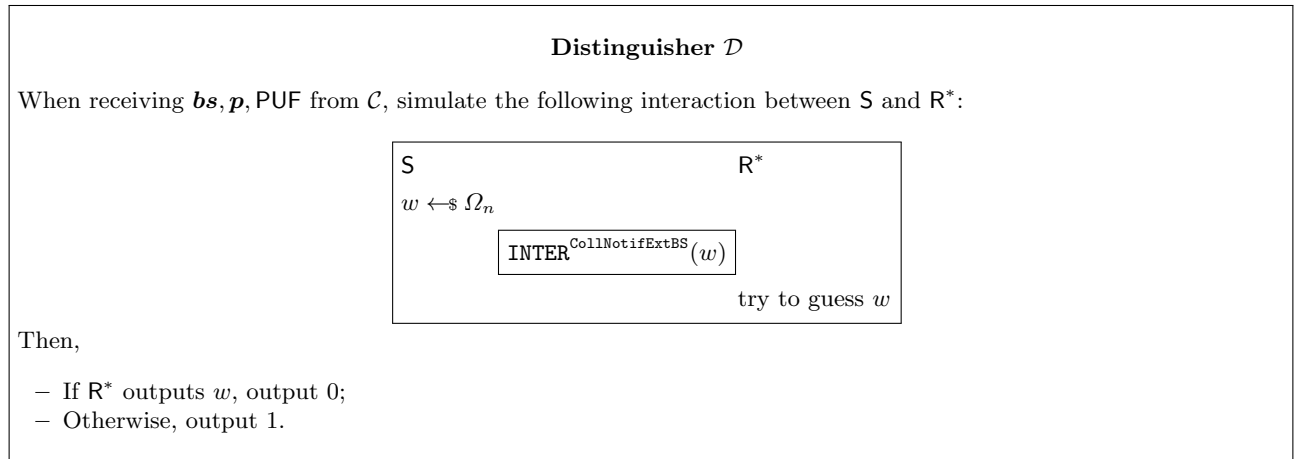


Fig. 34. Distinguisher that breaks the generalized indistinguishability property depicted in Fig. 22.

like in Theorem 2, we want to show that

$$\Pr[R_{I_J}^I = \text{Dec}(Q_{X^I})_{I_J} \oplus \text{Dec}(Q_{Y^I})_{I_J}]$$

is negligible. Notice that

$$\begin{aligned} \tilde{H}_\infty(R_{I_J}^I \mid \text{Dec}(Q_{X^I})_{I_J} \oplus \text{Dec}(Q_{Y^I})_{I_J}) &\geq \tilde{H}_\infty(R_{I_J}^I \mid Q_{X^I}, Q_{Y^I}, J) && \text{(Lemma 4)} \\ &\geq \tilde{H}_\infty(R_{I_J}^I \mid Q_{X^I}, Q_{Y^I}) - H_0(J) && \text{(Lemma 7)} \\ &\geq n - H_0(I) - H_0(J) - H_0(J) && \text{(Lemma 6)} \\ &= n - \log(N(n)) - 2\log(k(n)). \end{aligned}$$

Thus, by Lemma 8,

$$\Pr[R_{I_J}^I = \text{Dec}(Q_{X^I})_{I_J} \oplus \text{Dec}(Q_{Y^I})_{I_J}] \leq 2^{-n+\log(N(n))+2\log(k(n))},$$

which is negligible, since $N(n)$ and $k(n)$ are polynomial.

Extractability: Here, we use the extractor from Theorem 2 for each string being committed, as depicted in Fig. 35. Notice that for each $i \in [N]$, only PUF_E^i is considered when extracting x^i . The proof is completely analogous to what was done in Theorem 2. Indeed, as the extractability proof of ExtPUF relied on its binding proof, the same happens for CollNotifExtPUF .

Extractor E

E proceeds like an honest R, while also saving S^* 's queries to PUF_E^i in $\mathcal{Q}$. Once the commitment phase is completed, it does the following:

- If S^* queried any PUF_E^i after receiving the corresponding r^i , output $x^* = \perp$;
- Otherwise, for each for each $i \in [N]$ and $q \in \mathcal{Q}$, try to extract a string x^i from c^i and q by running $\text{ExtractFromQuery}^i(\text{Dec}(q))$, where:

ExtractFromQueryⁱ(st)

$x^i := \varepsilon$

for $j \in [k]$:

if $c_{I_j}^i = st_{I_j} \wedge c_{I_j}^i \neq st_{I_j} \oplus r_{I_j}^i$:

$x^i := x^i \parallel 0$

elseif $c_{I_j}^i = st_{I_j} \oplus r_{I_j}^i \wedge c_{I_j}^i \neq st_{I_j}$:

$x^i := x^i \parallel 1$

else :

return $\perp$

return x^i

Then, output x^* , where for each $i \in [N]$:

- If exactly one string x^i was extracted from c^i , $x^{*i} := x^i$;
- Otherwise, $x^{*i} := \perp$.

Fig. 35. The extractor for CollNotifExtPUF .

The collective version of StatelessExtPUF , $\text{CollStatelessExtPUF}$, is depicted in Fig. 36.

Theorem 6. *CollStatelessExtPUF is an ideal extractable collective commitment in the $\mathcal{F}_{\text{MPUF}}$ -hybrid model with $k_{\text{state}} = 0$.*

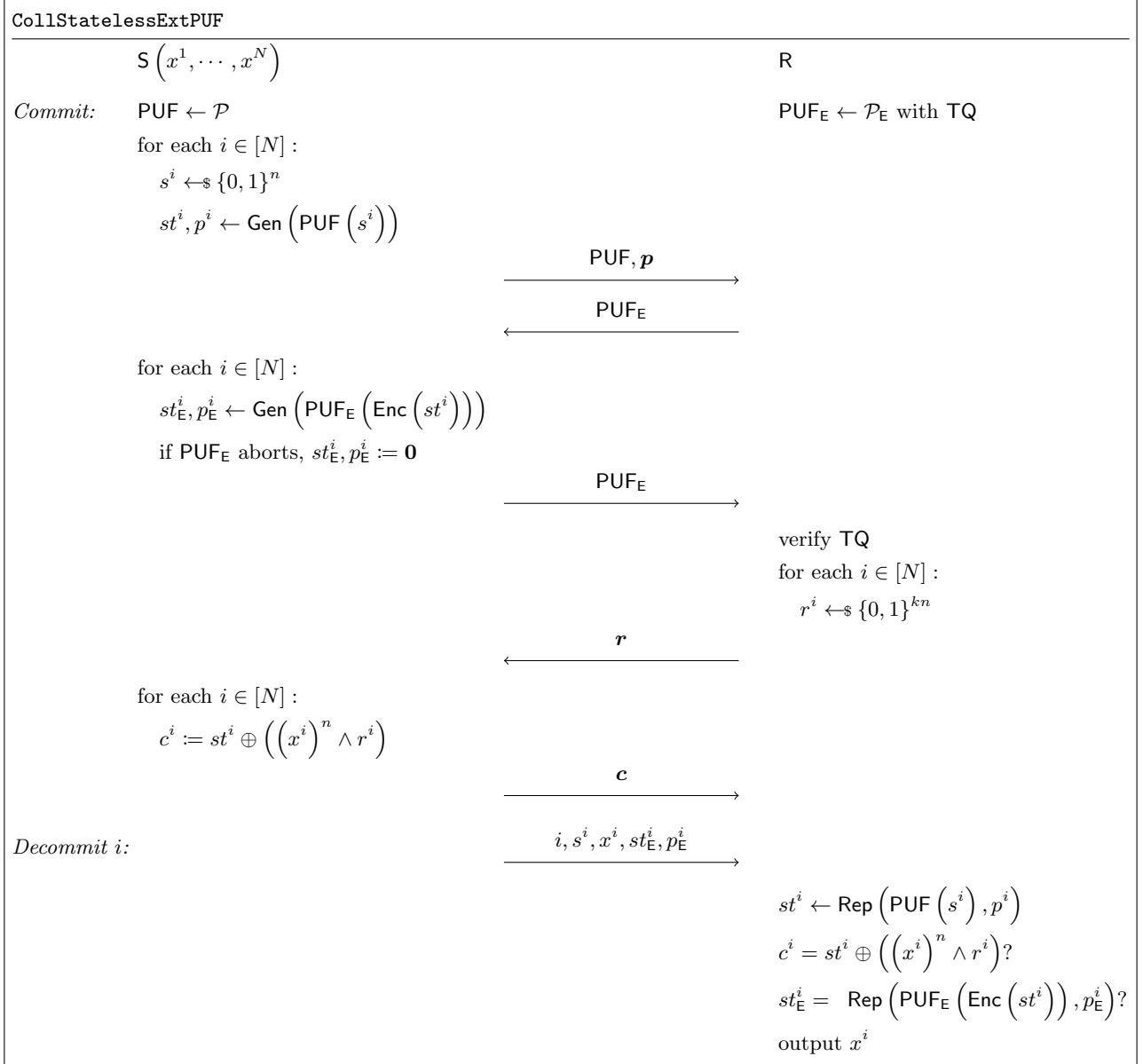


Fig. 36. The CollStatelessExtPUF protocol.

Proof. **Computationally Hiding:** This is essentially the same as in Theorem 5. The only difference is that now, S^* returns PUF_E during the commitment phase in **CollStatelessExtUnif**. However, since PUF_E is stateless, we can still ensure that R^* does not get any information that depends on W .

Statistically Binding: Analogous to Theorem 5.

Extractability: Analogous to Theorem 5. The extractor is depicted in Fig. 37. Notice that now the same PUF_E is considered for all $i \in [N]$.

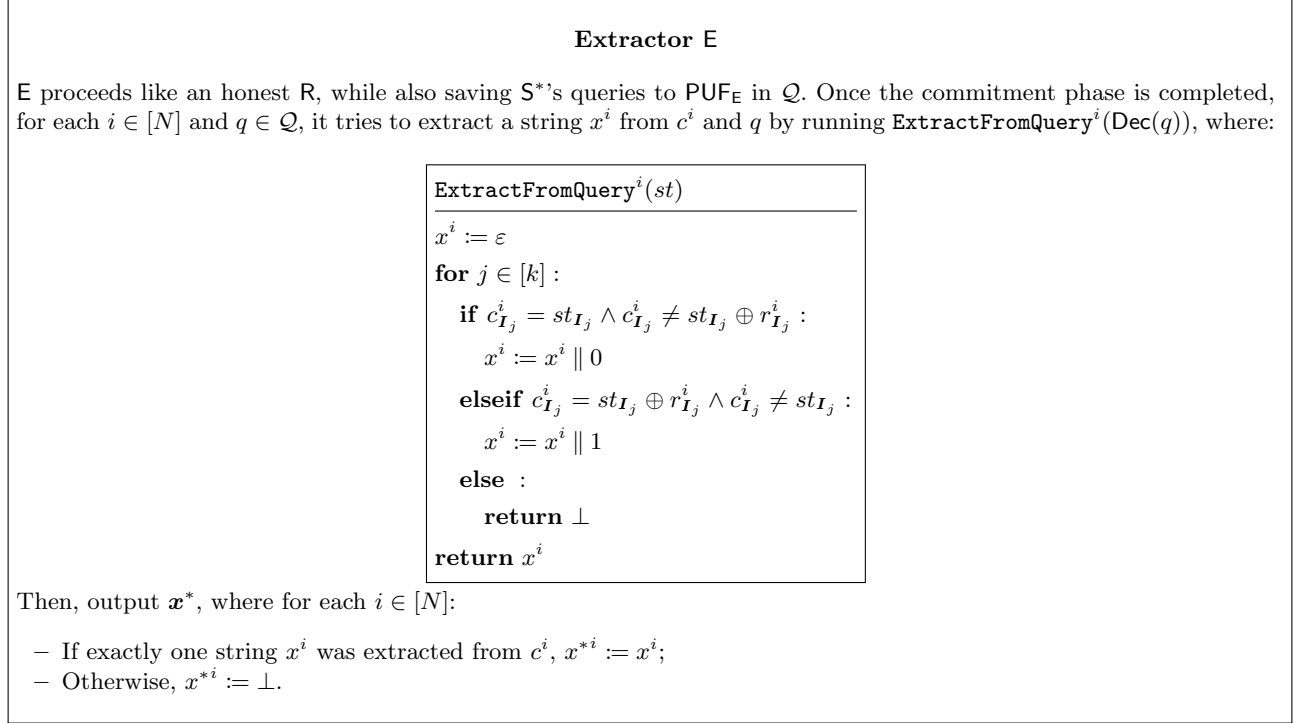


Fig. 37. The extractor for **CollStatelessExtPUF**.

F Proof of UC security

The bit commitment functionality is depicted in Fig. 38. Consider the protocol given by **UCCompiler** in Fig. 10

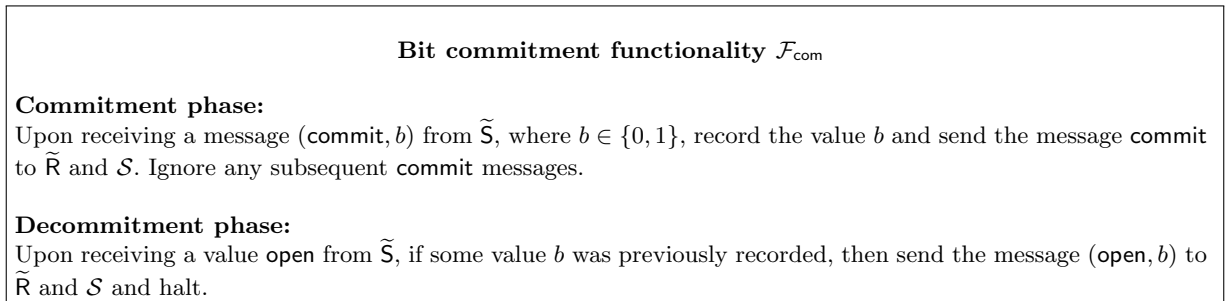


Fig. 38. Bit commitment functionality $\mathcal{F}_{\text{com}}$.

with an ideal extractable collective commitment **CollCom**. We prove that this protocol UC-realizes $\mathcal{F}_{\text{com}}$.

Lemma 10. *Let CollCom be an ideal extractable collective commitment scheme in the $\mathcal{F}_{\text{MPUF}}$ -hybrid model. Then, for any real world adversary $\mathcal{A}$ that corrupts the receiver, there exists a simulator $\mathcal{S}$ such that no environment $\mathcal{Z}$ can distinguish between the corresponding real world and ideal world processes.*

Proof. Consider the simulator defined in Fig. 39. We aim to show that no environment $\mathcal{Z}$ can distinguish the real world from the ideal world by using a hybrid argument, just like in [12]. Starting from the real world, we define a sequence of hybrids that gradually transition to the ideal world. Each intermediate hybrid is defined within a modified version of the ideal world, where the simulator has access to the input $b_{\mathcal{Z}}$ chosen by $\mathcal{Z}$ for $\mathcal{S}$. Furthermore, we construct each hybrid's simulator based on the previous one and prove that $\mathcal{Z}$ cannot distinguish between consecutive hybrids. Since the final hybrid corresponds to the ideal world, using the simulator defined in Fig. 39, the result follows.

Simulator $\mathcal{S}$

After being notified by $\mathcal{F}_{\text{com}}$ that $\mathcal{Z}$ has given the secret bit as input to $\tilde{\mathcal{S}}$, the following interaction between $\mathcal{S}$ and $\mathcal{R}^*$, which is controlled by $\mathcal{A}$, is simulated:

- **Commit:**
 1. $\mathcal{R}^*$ interacts with $\mathcal{S}$ to make a certain commitment using CollCom. In that interaction, let $\mathcal{S}$ run the corresponding extractor to obtain $\mathbf{e}^* := (e_i^*)_{i \in [n, 2]} \in \{0, 1, \perp\}^n$.
 2. For each $i \in [n, 2]$:
 - Sample $(l_i^0, l_i^1) \leftarrow \{(i, i+1), (i+1, i)\}$;
 - Sample $b_{l_i^0}^0 \leftarrow \{0, 1\}$ and let $b_{l_i^0}^1 := b_{l_i^0}^0$. These bits correspond to a blob of 0;
 - Sample $b_{l_i^1}^0 \leftarrow \{0, 1\}$ and $b_{l_i^1}^1 := \neg b_{l_i^1}^0$. These bits correspond to a blob of 1.
 3. Let $\mathcal{S}$ commit to $(b_j^0, b_j^1)_{j \in [2n]}$ using CollCom.
 4. For each $i \in [n, 2]$:
 - If $e_i^* = \perp$, let $y_i \leftarrow \{0, 1\}$;
 - Otherwise, let $y_i := b_{l_i^0}^{e_i^*} \oplus b_{l_i^1}^{e_i^*}$.
 5. Simulate a modified version of BlobEqualities between $\mathcal{S}$ and $\mathcal{R}^*$, where $\mathcal{S}$ sends $\mathbf{y} := (y_i)_{i \in [n, 2]}$ instead of the one specified in the protocol. If $\mathcal{R}^*$ is able to decommit to some $\mathbf{e} \neq \mathbf{e}^*$, abort.
- **Decommit:** When receiving $b_{\mathcal{Z}}$ from $\mathcal{F}_{\text{com}}$, let $\mathcal{S}$ open the blobs corresponding to $b_{\mathcal{Z}}$. That is, let $\mathcal{S}$ open $(b_{l_i^{b_{\mathcal{Z}}}}^0, b_{l_i^{b_{\mathcal{Z}}}}^1)_{i \in [n, 2]}$.

Fig. 39. Simulator for the case where the receiver is dishonest.

Consider the following hybrids:

- **Hybrid H_0 :** This is the real world execution of the protocol UCCompiler.
- **Hybrid H_1 :** This hybrid is in the modified ideal world defined above, where the simulator $\mathcal{S}_1$ simulates an execution of the real world process, using $b_{\mathcal{Z}}$ as input for $\mathcal{S}$. Since H_1 is just the real world process executed through the simulator $\mathcal{S}_1$, hybrids H_0 and H_1 are identical.
- **Hybrid H_2 :** In this hybrid, consider the simulator $\mathcal{S}_2$ that runs steps 1 and 5 of $\mathcal{S}$. Notice that, from the extractability property of CollCom, the extractor simulates an honest receiver (which, in this case, is $\mathcal{S}$) and extracts $\mathbf{e}^*$ such that $\mathcal{R}^*$ is only able to decommit to some $\mathbf{e} \neq \mathbf{e}^*$ with negligible probability. Furthermore, since for each $i \in [n, 2]$, the blobs $\mathbf{B}_i$ and $\mathbf{B}_{i+1}$ have the same value, we have $b_i^{e_i^*} \oplus b_{i+1}^{e_{i+1}^*} = b_i^0 \oplus b_{i+1}^0$. Thus, this is indistinguishable to H_1 .
- **Hybrid H_3 :** In this hybrid, consider the simulator $\mathcal{S}_3$ that generates its l_i^0 and l_{i+1}^1 for each $i \in [n, 2]$ during the commitment phase. Furthermore, in the decommitment phase, $\mathcal{S}_3$ lets $\mathcal{S}$ open the blobs of indices $l_i^{b_{\mathcal{Z}}}$. This is identical to H_2 .
- **Hybrid H_4 :** In this hybrid, consider the simulator $\mathcal{S}$ that we defined earlier. Notice that we are now in the regular ideal world, where $\mathcal{S}$ no longer has access to $b_{\mathcal{Z}}$. This hybrid is not identical to H_3 , since the blobs no longer have the same value. However, we will show that no environment $\mathcal{Z}$ can distinguish between them. Consider the interaction depicted in Fig. 40 between a challenger $\mathcal{C}$ and $\mathcal{Z}$, in which $\mathcal{Z}$ attempts to distinguish between H_3 and H_4 .

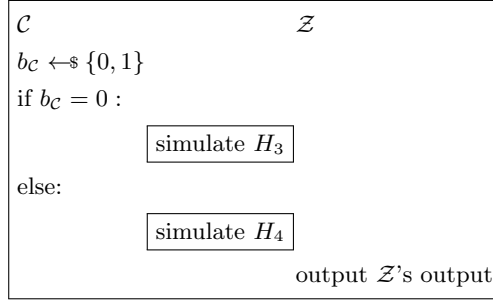


Fig. 40. Interaction corresponding to indistinguishability between hybrids H_3 and H_4 .

Assume, by contradiction, that there exists $\mathcal{Z}$ such that $\Pr[\mathcal{Z} = B_C] - \frac{1}{2}$ is non-negligible. Since the only difference between the two hybrids lies in the values of the bits being committed by $\mathcal{S}$, we will be able to construct an interaction that breaks the computational hiding property of CollCom , thus showing that both hybrids are indeed indistinguishable.

Consider the interaction depicted in Fig. 41 between $\mathcal{S}$ and $\mathcal{R}^*$, where **INTER** is defined in Fig. 42. Notice

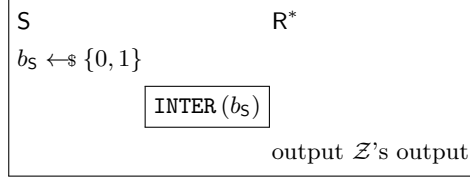


Fig. 41. Reduction to the computational hiding property of CollCom .

how important it is for $\mathcal{R}^*$ to commit before $\mathcal{S}$, since the bits $\mathcal{S}$ commits to depend on what it extracts from $\mathcal{R}^*$'s commitment. Furthermore, notice that for each value of b_S , the only thing that differs is the value of the blobs. Indeed, when $b_S = 0$, all the blobs have value b_Z , causing the simulation to be distributed like in H_3 . On the other hand, when $b_S = 1$, there are blobs of 0 and 1, and so it is distributed like in H_4 . Therefore, the probability of $\mathcal{R}^*$ correctly guessing b_S is

$$\begin{aligned}
 2 \Pr[\mathcal{R}^*(\text{INTER}(B_S)) = B_S] &= \Pr[\mathcal{R}^*(\text{INTER}(0)) = 0 \mid B_S = 0] + \Pr[\mathcal{R}^*(\text{INTER}(1)) = 1 \mid B_S = 1] \\
 &= \Pr[\mathcal{Z} = 0 \mid B_C = 0] + \Pr[\mathcal{Z} = 1 \mid B_C = 1] \\
 &= 2 \Pr[\mathcal{Z} = B_C]
 \end{aligned}$$

and thus

$$\Pr[\mathcal{R}^*(\text{INTER}(B_S)) = B_S] - \frac{1}{2} = \Pr[\mathcal{Z} = B_C] - \frac{1}{2},$$

which we assumed to be non-negligible. This contradicts the computational hiding property of CollCom .

Lemma 11. *Let CollCom be an ideal extractable collective commitment scheme in the $\mathcal{F}_{\text{MPUF}}$ -hybrid model. Then, for any real world adversary $\mathcal{A}$ that corrupts the sender, there exists a simulator $\mathcal{S}$ such that no environment $\mathcal{Z}$ can distinguish between the corresponding real world and ideal world processes.*

Proof. Consider the simulator defined in Fig. 43. Notice that the real and ideal world processes are almost identical. Indeed, $\mathcal{S}$ lets $\mathcal{R}$ run the extractor for CollCom , which simulates an honest receiver (which in this case is $\mathcal{R}$), and then it lets $\mathcal{R}$ honestly follow the protocol **BlobEqualities**. The only difference is that it additionally aborts in the following cases:

- In step 6, if $A = \{0,1\}$, meaning $A_i = \{0,1\}$ for all $i \in [n,2]$. This means that $\mathcal{S}^*$ managed to cheat the **BlobEqualities** protocol, which we are going to show only happens with negligible probability. Due to extractability property of CollCom , we know that with overwhelming probability, the commitments of each b_j^e can only be opened to b_j^{*e} . Consider the function $f : \{0,1\}^n \rightarrow \{0,1\}^n$ such that $f(e) = (f_i(e_i))_{i \in [n,2]} =$

INTER

- $\mathcal{Z}$ gives some input b_Z to $\mathcal{S}$.
- **Commit:**
 1. $\mathcal{R}^*$ interacts with $\mathcal{S}$ to make a certain commitment using **CollCom**. In that interaction, $\mathcal{S}$ runs the corresponding extractor to obtain $\mathbf{e}^* := (e_i^*)_{i \in [n, 2]} \in \{0, 1, \perp\}^n$.
 2. For each $i \in [n, 2]$, $\mathcal{S}$ does:
 - * Sample $(\alpha_i, \beta_i) \leftarrow \{(i, i+1), (i+1, i)\}$;
 - * Sample $b_{\alpha_i}^0 \leftarrow \{0, 1\}$ and let $b_{\alpha_i}^1 := b_{\alpha_i}^{e_i^*} \oplus b_Z$. These bits correspond to a blob of b_Z ;
 - * Sample $b_{\beta_i}^{e_i^*} \leftarrow \{0, 1\}$ and let $b_{\beta_i}^{\neg e_i^*} := b_{\beta_i}^{e_i^*} \oplus b_Z \oplus b_S$. These bits correspond to a blob of $b_Z \oplus b_S$.
 3. $\mathcal{S}$ defines
 - * the function $\mathbf{STRINGS}_n : \{0, 1\} \rightarrow \{0, 1\}^{4n}$ as $\mathbf{STRINGS}_n(b_S) = (b_j^0, b_j^1)_{j \in [2n]}$ defined above;
 - * the set $\mathbf{OPEN}_n$ containing the indices of all the bits $b_{\alpha_i}^0, b_{\alpha_i}^1, b_{\beta_i}^{e_i^*}$ for each $i \in [n, 2]$ (notice that none of these bits depend on b_S),
and commits to $\mathbf{STRINGS}_n(b_S)$.
 4. For each $i \in [n, 2]$, $\mathcal{S}$ does:
 - * If $e_i^* = \perp$, let $y_i \leftarrow \{0, 1\}$;
 - * Otherwise, let $y_i := b_{\alpha_i}^{e_i^*} \oplus b_{\beta_{i+1}}^{e_{i+1}^*}$.
 5. Run a modified version of **BlobEqualities**, where $\mathcal{S}$ sends $\mathbf{y} := (y_i)_{i \in [n, 2]}$ instead of the one specified in the protocol. If $\mathcal{R}^*$ is able to decommit to some $\mathbf{e} \neq \mathbf{e}^*$, $\mathcal{S}$ aborts.
- **Decommit:** $\mathcal{S}$ opens the blobs corresponding to b_Z . That is, $\mathcal{S}$ opens $(b_{\alpha_i}^0, b_{\alpha_i}^1)_{i \in [n, 2]}$.

Fig. 42. Definition of the interaction **INTER**.

Simulator $\mathcal{S}$

Simulate the interaction between $\mathcal{S}^*$ and $\mathcal{R}$ as follows:

- **Commit:**
 1. For each $i \in [n, 2]$:
 - Sample $e_i \leftarrow \{0, 1\}$.
 2. Let $\mathcal{R}$ commit to $\mathbf{e} := (e_i)_{i \in [n, 2]}$ using **CollCom**.
 3. $\mathcal{S}^*$ interacts with $\mathcal{R}$ to make a certain commitment using **CollCom**. In that interaction, let $\mathcal{R}$ run the corresponding extractor to obtain $(b_j^{*0}, b_j^{*1})_{j \in [2n]} \in \{0, 1, \perp\}^{2n}$.
 4. For each $j \in [2n]$:
 - If $b_j^{*0} = \perp$ or $b_j^{*1} = \perp$, let $b_j^* := \perp$;
 - Otherwise, let $b_j^* := b_j^{*0} \oplus b_j^{*1}$.
 5. Simulate **BlobEqualities** between $\mathcal{S}^*$ and $\mathcal{R}$.
 6. Consider the sets $A_i := \{b_i^*, b_{i+1}^*\}$ for each $i \in [n, 2]$ and let $A := \bigcap_{i \in [n, 2]} A_i \setminus \{\perp\}$. This set represents to which bits $\mathcal{S}^*$ can decommit to in the decommitment phase. Check which of the following cases applies:
 - (a) If $A = \emptyset$, let $\tilde{\mathcal{S}}$ send $b^* := 0$ to $\mathcal{F}_{\text{com}}$;
 - (b) If $A = \{b^*\}$ for some b^* , let $\tilde{\mathcal{S}}$ send b^* to $\mathcal{F}_{\text{com}}$;
 - (c) If $A = \{0, 1\}$, abort.
- **Decommit:** If $\mathcal{S}^*$ correctly decommits to some b , then
 - If $b = b^*$, let $\tilde{\mathcal{S}}$ send open to $\mathcal{F}_{\text{com}}$;
 - Otherwise, abort.

Fig. 43. Simulator for the case where the sender is dishonest.

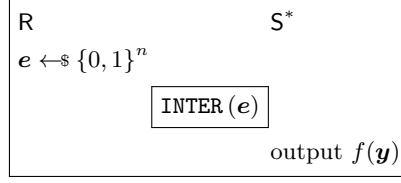


Fig. 44. Reduction to the computational hiding property of `CollCom`.

$(b_i^{*e_i} \oplus b_{i+1}^{*e_i})_{i \in [n, 2]}$, where $\mathbf{e}$ is indexed in $[n, 2]$. Notice that for each $i \in [n, 2]$

$$A_i = \{b_i^{*0} \oplus b_i^{*1}, b_{i+1}^{*0} \oplus b_{i+1}^{*1}\} = \{0, 1\}.$$

Furthermore,

$$\begin{aligned}
\forall i \in [n, 2] \quad \{b_i^{*0} \oplus b_i^{*1}, b_{i+1}^{*0} \oplus b_{i+1}^{*1}\} = \{0, 1\} &\iff \forall i \in [n, 2] \quad b_i^{*0} \oplus b_i^{*1} \neq b_{i+1}^{*0} \oplus b_{i+1}^{*1} \\
&\iff \forall i \in [n, 2] \quad b_i^{*0} \oplus b_i^{*0} \neq b_{i+1}^{*1} \oplus b_{i+1}^{*1} \\
&\iff \forall i \in [n, 2] \quad f_i(0) \neq f_i(1) \\
&\iff \forall i \in [n, 2] \quad f_i \text{ is a bijection in } \{0, 1\} \\
&\implies \forall i \in [n, 2] \quad f_i \circ f_i = \text{id}_{\{0, 1\}} \implies f \circ f = \text{id}_{\{0, 1\}^n}.
\end{aligned}$$

Now, observe that S^* successfully passed `BlobEqualities` and the commitments of each b_j^e were opened to b_j^{*e} . This means that for each $i \in [n, 2]$ it sent $y_i = b_i^{*e_i} \oplus b_{i+1}^{*e_i} = f_i(e_i)$. In other words, it sent $\mathbf{y} = f(\mathbf{e})$, and since $f \circ f = \text{id}_{\{0, 1\}^n}$, we know $\mathbf{y} = f(\mathbf{e}) \iff f(\mathbf{y}) = \mathbf{e}$.

This implies that S^* could successfully guess the $\mathbf{e}$ generated by R . However, this only happens with negligible probability due to the computational hiding property of `CollCom`.

More specifically, consider the interaction depicted in Fig. 44 between R and S^* , where `INTER` denotes an execution of `UCCompiler` in which R commits to $\mathbf{e}$ using `CollCom` and $\mathbf{y}$ denotes the one S^* sends in `BlobEqualities`. Then, the probability that the simulation aborted in this step is

$$\Pr[\mathbf{Y} = f(\mathbf{E})] = \Pr[f(\mathbf{Y}) = \mathbf{E}] = \Pr[\text{S}^*(\text{INTER}(\mathbf{E})) = \mathbf{E}],$$

which is negligible by the computational hiding property of `CollCom`.

- In the decommitment phase, when S^* opens $b \neq b^*$. Due to the extractability property of `CollCom`, this only happens with negligible probability.

Thus, we conclude that the simulator only additionally aborts with negligible probability. This, along with the fact that the interaction simulated by $\mathcal{S}$ is otherwise identical to the real-world process, implies that no environment $\mathcal{Z}$ can distinguish between the real and ideal worlds.

Thus, from Lemmas 10 and 11, we can finally conclude the following:

Theorem 4. *Let `CollCom` be an ideal extractable collective commitment scheme in the $\mathcal{F}_{\text{MPUF}}$ -hybrid model. Then, the corresponding protocol given by `UCCompiler` UC-realizes $\mathcal{F}_{\text{com}}$ in the $\mathcal{F}_{\text{MPUF}}$ -hybrid model.*